Nature-Inspired Computing

Study Text for the Final State Examination

Final State Examination in Artificial Intelligence, MFF UK

August 2026

This text covers the examination topic *Nature-Inspired Computing* of the master's final state examination in Artificial Intelligence at MFF UK. Its chapters correspond **one to one** to the sentences of the official wording of the topic (see the mapping table below). It is based on the slides of the lecture courses NAIL025 (Evolutionary Algorithms I) and NAIL086 (Evolutionary Algorithms II), supplemented with the standard material of the textbooks Eiben & Smith: *Introduction to Evolutionary Computing*, Mitchell: *An Introduction to Genetic Algorithms*, Michalewicz: *Genetic Algorithms + Data Structures = Evolution Programs* and Poli, Langdon & McPhee: *A Field Guide to Genetic Programming*. It contains definitions, pseudocode, derivations, numerical examples and comparison tables; every chapter ends with a summary for the oral examination. This is an English translation of the Czech study text *Přírodou inspirované počítání*.

Scope and marking

The length of the chapters reflects the weight of the material: most space goes to genetic algorithms with genetic programming, to schema theory, and to the applications — that is, to what the lectures treat in most detail.

The marker [♦ beyond the slides] flags material that is **in none of the available slide decks** — neither EVA I, nor EVA II, nor the optional *Evolutionary Robotics* (Mráz) — but has been kept in the text, either because *the official wording of the topic names it*, or because the surrounding exposition would not make sense without it. It is an offer for deletion: whatever is not marked is backed by the slides. Where the material is only mentioned in passing, the marker says how briefly.

The most conspicuous case is **open-ended evolution**: the topic names it explicitly, the slides give it a single sentence. Similarly ant colony algorithms (ACO) — the topic speaks of swarm algorithms in the plural, the slides cover only PSO (the swarm *robotics* in Evolutionary Robotics is a different thing from swarm optimisation). Conversely, *grammatical evolution* and *interactive evolution* are *not* marked: they are absent from EVA, but Evolutionary Robotics covers them.

Mapping of the examination topic to chapters

Sentence of the official topic	Chapter in this text
Genetic algorithms, genetic and evolutionary programming.	1 Genetic algorithms, genetic and evolutionary programming
Schema theory, probabilistic models of the simple genetic algorithm.	2 Schema theory and probabilistic models of the SGA
Evolution strategies, differential evolution, coevolution, open-ended evolution.	3 Evolution strategies, differential evolution, coevolution, open-ended evolution
Swarm optimisation algorithms.	4 Swarm optimisation algorithms
Memetic algorithms, hill climbing, simulated annealing.	5 Local search and memetic algorithms
Applications of evolutionary algorithms (evolution of expert systems, neuroevolution, solving combinatorial problems, multi-objective optimisation).	6 Applications of evolutionary algorithms

Contents

1 Genetic algorithms, genetic and evolutionary programming

The longest sentence of the topic; the chapter therefore starts with the general scheme of an evolutionary algorithm (without which the rest of the text makes no sense), continues with Holland's simple genetic algorithm and the representations of an individual, and ends with genetic and evolutionary programming, that is, the evolution of executable structures.

1.1 The general scheme of an evolutionary algorithm

1.1.1 Placing the field

Nature-inspired computing is an umbrella term for metaheuristic algorithms that borrow principles observed in living (and sometimes non-living) nature. The traditional division is as follows:

- **Computational intelligence** (formerly *soft computing*) covers neural networks, fuzzy systems and evolutionary computation. It stands in opposition to "hard", logically and symbolically oriented AI.
- **Evolutionary computation** (EC) is a superset that also includes methods which are not evolutionary in the narrow sense (swarm algorithms, memetic algorithms, tabu search).
- **Evolutionary algorithms** (EA) are the subset of EC that uses the basic principles of natural evolution: a population of candidate solutions, stochastic operators (crossover, mutation) and selection driven by an objective function.

EAs are **population-based stochastic search algorithms**. They are used where no gradient or analytical form of the objective function is available (so-called *black-box optimisation*), where the solution space is discrete or structured (trees, graphs, permutations), or where several mutually non-dominated solutions are sought at once.

1.1.2 Biological inspiration

- **Darwin (1859, On the Origin of Species).** The environment has limited resources, reproduction is the key to survival, and better adapted individuals have a greater chance of reproducing. Successful phenotypic traits are reproduced, modified and recombined.
- **Mendel (1866).** The gene as the basic unit of heredity; alleles are passed on independently. Reality is more complicated: *polygeny* (several genes influence one trait), *pleiotropy* (one gene influences several traits), mitochondrial DNA, epigenetics.
- **Watson and Crick (1953).** The structure of DNA; the molecular-biological view of how information is stored and inherited. A codon (a triple of nucleotides) encodes one of the amino acids, and the code is redundant.
- **Genotype → phenotype.** The mapping is one-way and complex (transcription DNA→RNA, translation RNA→protein). *Lamarckism*, by contrast, assumes the existence of an inverse mapping, i.e. the inheritance of acquired characteristics. It is rejected in biology, but it is used as a technique in memetic algorithms (chapter 5).

Glossary: nature vs. algorithm

environment	optimisation problem
individual / chromosome	candidate solution, point of the search space
gene, allele	component of a solution and its value
genotype	internal representation (encoding) of an individual
phenotype	the "outside" of an individual, on which quality is evaluated
fitness	measure of solution quality (objective function)
population	multiset of candidate solutions
generation	one iteration of the algorithm

1.1.3 The general scheme of an evolutionary algorithm

Evolutionary algorithm

An evolutionary algorithm is an iterative stochastic process that maintains a population $P(t)$ of candidate solutions and repeatedly creates from it the population $P(t+1)$ by means of *parent selection*, *recombination*, *mutation* and *environmental selection*, until a termination condition is met.

Algorithm 1 General scheme of an evolutionary algorithm

Input: representation, fitness f, operators, parameters

1: $t \leftarrow 0$; initialise the population $P(0)$ at random
2: evaluate all individuals in $P(0)$ with the function f
3: **while** the termination condition is not met **do**
4: **parent selection:** select a set of parents from $P(t)$ (probabilistically, according to fitness)
5: **recombination:** from pairs (n-tuples) of parents create offspring with probability p_C
6: **mutation:** perturb each offspring at random with probability p_M
7: evaluate the offspring $P'(t+1)$
8: **environmental selection:** from $P(t) \cup P'(t+1)$ (or from $P'(t+1)$ only) select the new population $P(t+1)$
9: $t \leftarrow t+1$
10: **end while**

Output: the best individual found

This cycle is common to all EAs; the individual variants (GA, ES, EP, GP, DE) differ in the choice of representation, in the emphasis placed on the individual operators and in the type of selection. Two opposing forces play the key role:

- **Variation** (crossover and mutation) creates new diversity — it provides *exploration* of the space.
- **Selection** removes diversity and steers the search towards promising regions — it provides *exploitation*.

The balance between them is the central problem of EA design. Exploration alone degenerates into a random walk; exploitation alone leads to getting stuck in a local optimum and to *premature convergence*.

1.1.4 Components of an evolutionary algorithm

Representation. The choice of encoding is the most important design decision. The classical options are: a binary string, a vector of integers, a vector of real numbers, a permutation, a tree, a graph, a matrix, a finite automaton, a neural network. The representation determines which operators make sense.

Fitness function. A mapping f : phenotype space $\to \mathbb{R}$ that measures quality. Minimisation problems are usually converted to maximisation (e.g. $f' = C_{\max} - f$ or $f' = 1/(1+f)$). Fitness may be *deterministic*, *noisy* (simulation), *dynamic* (changing in time, section 3.3) or *contextual* (dependent on the other individuals — coevolution).

Population. A multiset of individuals of (usually) fixed size n. The population is the carrier of *diversity*; diversity is measured, for example, by the average Hamming distance, by the entropy of the alleles at the individual positions, or by the variance of fitness.

Initialisation. Usually uniformly random. The alternatives are *seeding* (inserting heuristic or expert solutions — with the risk of premature convergence), constructive heuristics (e.g. nearest neighbour for the TSP), and Latin hypercubes for real-valued vectors.

Termination condition. A maximum number of generations or fitness evaluations, reaching a target quality, stagnation of the best fitness for k generations, or loss of population diversity.

1.1.5 Selection mechanisms

Roulette wheel (fitness-proportionate) selection

Roulette wheel selection

The individual x_i is selected with probability

$$p_i = \frac{f(x_i)}{\sum_{j=1}^{n} f(x_j)} = \frac{f(x_i)}{n\,\bar{f}},$$

where $\bar{f}$ is the average fitness of the population. The expected number of times an individual is selected in n draws is $np_i = f(x_i)/\bar{f}$. This is sampling *with replacement* — one individual may be selected several times.

Example (Goldberg's classical one). A population of four 5-bit strings, fitness $f(x) = x^2$ (decoded as an integer):

i	string	x	$f = x^2$	$p_i = f/\sum f$	exp. count $f/\bar{f}$
1	01101	13	169	0.144	0.58
2	11000	24	576	0.492	1.97
3	01000	8	64	0.055	0.22
4	10011	19	361	0.309	1.23
	sum		1170	1.000	4.00
	average $\bar{f}$		292.5		

The roulette wheel divides the circumference into sectors of sizes p_i and is spun n times. Individual 2 will occupy roughly two places in the mating pool, individual 3 will probably drop out.

Disadvantages of roulette wheel selection:

- *Premature dominance* of a "superman": an individual with extreme fitness takes over the population and diversity disappears.
- *Loss of selection pressure* towards the end of the run: when all fitness values are similar, selection approaches a random draw.
- It requires positive fitness values and is sensitive to affine transformations (f and $f + c$ give different probabilities). The remedy is *scaling* (section 1.2.5).
- High variance of sampling — the actual counts may differ considerably from the expected ones.

Stochastic universal sampling (SUS) Stochastic universal sampling uses a single spin of the wheel with n evenly spaced pointers at a distance of $1/n$. The expected numbers of selections remain the same as for the roulette wheel, but the variance is minimal: an individual with expected count e_i is selected $\lfloor e_i \rfloor$ or $\lceil e_i \rceil$ times. It is therefore a "fairer roulette wheel".

Tournament selection

k-tournament selection

Select k individuals at random (uniformly, with replacement) and return the best of them. The parameter k (typically 2, 3, 5) directly controls the selection pressure: the probability that the i-th best individual out of n is selected (*here* $i = 1$ denotes the **best** one) is $\big((n-i+1)^k - (n-i)^k\big)/n^k$ — it is the probability that all k draws fall among the i-th individual and the worse ones, minus the probability that all k fall strictly below the i-th. A deterministic tournament can be softened: the better individual wins with probability $p < 1$.

Advantages: it requires no global information (no summation of fitness), it is invariant with respect to monotone transformations of fitness, it is easy to parallelise, and it works even where fitness is not given explicitly and the comparison is obtained from *playing* (a played match, a simulation, coevolution).

Rank-based selection Rank-based selection replaces fitness by the rank of the individual. Linear rank selection assigns to the individual of rank i (*note, here the ordering is the opposite of the tournament case:* worst $i = 1$, best $i = n$) the probability

$$p_i = \frac{1}{n}\Big(s^- + (s^+ - s^-)\,\frac{i-1}{n-1}\Big), \qquad 1 \le s^+ \le 2, \quad s^- = 2 - s^+,$$

where s^+ is the expected number of copies of the best individual. The exponential variant uses $p_i \propto (1-c)c^{n-i}$. Rank-based selection maintains a constant selection pressure throughout the whole run and is robust both against "supermen" and against a change of the fitness scale.

Boltzmann selection The selection probability is $p_i \propto e^{f(x_i)/T}$ with a *temperature* T that decreases in time: at the beginning (high T) selection is almost uniform (exploration), at the end (low T) almost deterministic (exploitation). Analogously the *Boltzmann tournament*: the worse individual wins with a probability depending exponentially on the ratio of the fitness difference to the temperature. There is a direct link to simulated annealing (chapter 5).

Environmental selection, elitism, population models

- **Generational model**: the whole population is replaced by the offspring, $|P'| = |P| = n$.
- **Steady-state model**: in each step only λ (often 1–2) offspring are created, and they replace selected individuals (the worst one, a random one, the most similar one — *crowding*). The parameter *generation gap* $G = \lambda/n$ gives the proportion of the population that is replaced.
- **Elitism**: the k best individuals are copied unconditionally into the next generation. This guarantees monotonicity of the best fitness found and is a necessary condition for a number of convergence theorems; too strong an elitism, however, reduces diversity.
- **Deterministic ES selection**: $(\mu + \lambda)$ selects the μ best individuals from parents and offspring together (implicit elitism), (μ, λ) from the offspring only (chapter 3).

Selection pressure and takeover time

Selection pressure is the degree to which better individuals are favoured. It is quantified by the *takeover time* τ^* — the number of generations in which a single copy of the best individual (without variation) fills the whole population. For a tournament of size k we roughly have $\tau^* \approx \ln n / \ln k$; for fitness-proportionate selection the takeover time is longer and depends on the distribution of fitness.

1.1.6 Theoretical limits: No Free Lunch

Theorem 1.1 (No Free Lunch, Wolpert & Macready 1995–1997)**.** *Averaged over* all *possible objective functions on a finite domain, any two deterministic non-repeating search algorithms have the same expected performance. There is therefore no universally best optimisation algorithm.*

The practical consequence: it pays to build **domain-specific variants** of EAs — a suitable representation and operators that respect the structure of the problem. The theorem does not say that all algorithms are equally good on *real* problems; real functions form a very small and highly structured subset of all functions.

1.1.7 Historical branches of evolutionary algorithms

	GA	ES	EP	GP
author, year	Holland, 1975	Rechenberg, Schwefel, 1964	L. Fogel, Owens, Walsh, 1965	Koza, 1989–92
representation	binary string	vector $\mathbb{R}^n$ + strategy parameters	finite automaton (genotype = phenotype)	syntax tree (LISP)
main operator	crossover	mutation	mutation	subtree crossover
crossover	1-point	multi-parent recombination (local/global)	*none*	subtree exchange
mutation	bit-flip p_M	Gaussian, self-adaptive σ	"clever" automaton mutations	several types
parent selection	roulette wheel	random	each one once	tournament
env. selection	generational replacement	deterministic (μ, λ), $(\mu + \lambda)$	tournament (parents + offspring)	generational
theory	schemata	convergence, 1/5 rule	—	bloat, schemata

Today the boundaries between the branches are blurred (EP and ES, for instance, have practically merged) and we speak uniformly of evolutionary algorithms.

1.2 Genetic algorithms

1.2.1 The simple genetic algorithm (SGA)

Holland's original 1975 proposal is nowadays referred to as the *simple genetic algorithm* (SGA). It uses a minimal set of operators and the simplest encoding, and it was designed so as to make theoretical analysis possible (schema theory, chapter 2).

SGA

A population of n binary strings of length m; fitness-proportionate (roulette wheel) selection; one-point crossover with probability p_C; bit mutation with probability p_M per bit; generational replacement of the population ($P(t+1)$ replaces $P(t)$ entirely).

Algorithm 2 Simple genetic algorithm (SGA)

$t \leftarrow 0$; generate $P(0) = n$ strings of length m at random
while the termination condition is not met **do**
    compute $f(x)$ for every $x \in P(t)$
    $P(t+1) \leftarrow \emptyset$
    for $n/2$ times **do**
        select a pair of parents x, y from $P(t)$ by the roulette wheel
        with probability p_C cross x, y by one-point crossover
        flip every bit of x and y with probability p_M
        insert x, y into $P(t+1)$
    end for
    $t \leftarrow t + 1$
end while

Typical parameter values: $n \in [50, 500]$, $p_C \in [0.6, 0.9]$, $p_M \approx 1/m$ (i.e. on average one bit per individual is changed).

1.2.2 Binary representation

Encoding of integers and real numbers. A real number $x \in [a, b]$ encoded on k bits as an integer $z \in \{0, \ldots, 2^k - 1\}$ is decoded as

$$x \;=\; a \;+\; z \cdot \frac{b-a}{2^k - 1}.$$

The precision is $(b-a)/(2^k-1)$. Advantage: explicit control over the precision and compression of the search space. Disadvantage: *Hamming cliffs* — neighbouring values may differ in all bits (0111 $\rightarrow$ 1000), so a small change of the phenotype requires a large change of the genotype.

Gray code. It solves the Hamming cliffs: neighbouring integers differ in exactly one bit in the Gray code. Conversion from binary b to Gray g: $g_1 = b_1$, $g_i = b_{i-1} \oplus b_i$. Back: $b_1 = g_1$, $b_i = b_{i-1} \oplus g_i$.

Holland's argument for the binary alphabet. Binary strings of length 100 encode roughly the same information as decimal strings of length 30, but they contain more schemata (3^{100} against 11^{30}), so a GA "processes" more information. Today we know that schemata are not as fundamental as Holland believed, and that what matters is the **naturalness of the encoding for the given problem**. Real-valued vectors are used for real-valued optimisation, permutations for the TSP, and so on.

1.2.3 Crossover operators

Crossover (recombination)

An operator that creates offspring from two (or more) parents by combining their genetic material. In a GA it is the main operator; it expresses the hope that recombining good parts (*building blocks*) will produce even better individuals. It is applied with probability p_C, otherwise the offspring are copies of the parents.

One-point crossover. Choose a cut point $k \in \{1, \ldots, m-1\}$ at random and exchange the trailing parts.

Example. Parents $P_1 = 1011|0110$, $P_2 = 0010|1101$, cut point $k = 4$:

$$O_1 = \mathbf{1011}\,1101, \qquad O_2 = \mathbf{0010}\,0110.$$

Two-point crossover. Choose two points $k_1 < k_2$ and exchange the middle segment. The advantage over one-point crossover: the string behaves like a ring, so genes at the ends are not disadvantaged (with one-point crossover the probability of disrupting a segment is proportional to its *defining length*, see chapter 2).

Example. $P_1 = 10|1101|10$, $P_2 = 00|1011|01 \Rightarrow O_1 = 10\,1011\,10$, $O_2 = 00\,1101\,01$.

Uniform crossover. For each position j a coin is tossed: with probability 0.5 the gene comes from the first parent, otherwise from the second one (the second offspring is complementary). It does not favour adjacent positions, but it strongly disrupts schemata with a large defining length — it has a high *disruptiveness*.

Example. $P_1 = 11111111$, $P_2 = 00000000$, mask 10110010: $O_1 = 10110010$, $O_2 = 01001101$.

operator	prob. of disrupting a schema	note
one-point	$d/(m-1)$	positional bias, protects short schemata
two-point	$\dfrac{2d(m-d)}{m(m-1)}$	string as a ring, no end bias
k-point	grows with k	
uniform	$1-(1/2)^{o(S)-1}$	distributional bias, most disruptive

A remark on two-point crossover. If we regard the string as a ring, we choose two of the m cuts; a schema is disrupted if *exactly one* cut falls inside its defining segment of d gaps, i.e. with probability $d(m-d)/\binom{m}{2} = 2d(m-d)/(m(m-1))$. For a schema stretching from the first to the last bit ($d = m-1$) the disruption drops from certainty (one-point: $d/(m-1) = 1$) to a mere $2/m$ — this is exactly the "removal of the end bias". For short schemata, on the contrary, two-point crossover is about twice as destructive as one-point crossover.

1.2.4 Mutation and inversion

Bit mutation. Every bit is flipped independently with probability p_M. In the SGA mutation is a *secondary* operator — it acts as an insurance against getting stuck in a local optimum and against the permanent loss of an allele from the population. (In EP and in the early ES, by contrast, mutation is the only source of variability.) Recommendation: $p_M \approx 1/m$.

Inversion. Holland's original proposal also contained the operator of *inversion*: it reverses the order of a part of the string but *preserves the meaning of the bits* (a gene carries its own position identifier). The goal was to make it possible to evolve the ordering of the genes itself, so that cooperating genes lie close to one another and are not disrupted by crossover. This is technically demanding and the benefit was never demonstrated; with a permutation representation, however, inversion is used as a very effective mutation (section 1.3).

1.2.5 Fitness scaling

Roulette wheel selection is sensitive to the absolute values of fitness. At the beginning of a run the variance of fitness tends to be large (a few "supermen" take over the population), at the end small (selection loses pressure). The remedy is a transformation $f \mapsto f'$:

- **Linear scaling:** $f' = af + b$, where a, b are chosen so that $\bar{f}' = \bar{f}$ and $f'_{\max} = C_{\text{mult}} \cdot \bar{f}$, typically $C_{\text{mult}} \in [1.2, 2]$. A safeguard against negative values for the worst individuals is necessary.
- **Sigma truncation:** $f' = \max\big(0,\ f - (\bar{f} - c\,\sigma_f)\big)$, where σ_f is the standard deviation of fitness and $c \in [1, 3]$. It removes outlying low values and adapts automatically to the variance.
- **Power scaling:** $f' = f^{\,k}$, where k is increased during the run (the pressure grows).
- **Rank scaling:** replaces fitness by rank (see section 1.1.5) — the most robust and nowadays the most common option.
- **Boltzmann scaling:** $f' = e^{f/T}$ with a decreasing temperature.

Example. A population with fitness values $(169, 576, 64, 361)$, $\bar{f} = 292.5$, $f_{\max} = 576$. Linear scaling with $C_{\text{mult}} = 1.5$: we want $f'_{\max} = 438.75$ and $\bar{f}' = 292.5$. From the conditions $af_{\max} + b = 438.75$ and $a\bar{f} + b = 292.5$ we get $a = 146.25/283.5 = 0.516$, $b = 292.5 - 0.516 \cdot 292.5 = 141.6$. The new fitness values are (228.8; 438.8; 174.6; 327.9); the share of the best individual dropped from 49.2 % to 37.5 % — the selection pressure is moderated and the worst individual does not lose its chance.

1.3 Representation of an individual and genetic operators

The representation determines which operators make sense, and thereby also what the *fitness landscape* looks like — that is, how easy the search is. This is the most important practical decision when designing an EA.

1.3.1 Integer representation

An individual is a vector $(z_1, \ldots, z_m) \in \prod_j D_j$, where the D_j are finite domains. We distinguish:

- **Cardinal (ordinal) attributes** — a meaningful ordering and metric exist on the values (e.g. the number of machines, age).
- **Nominal attributes** — the values are mere codes (colour, type of component). Here it makes no sense to speak of "close" values.

Mutation.

- *Unbiased* (*random resetting*): the new value is drawn at random from the whole domain. The only correct choice for nominal attributes.
- *Biased* (*creep mutation*): the new value is the original one shifted by a small random step (e.g. from a discretised normal distribution). Suitable only for ordinal attributes.

Crossover. One-point, multi-point, uniform — exactly as with the binary representation, only integers are copied instead of bits.

1.3.2 Real-valued representation

An individual is $\vec{x} = (x_1, \ldots, x_n) \in \mathbb{R}^n$. Historically real numbers were encoded into bit strings; today one works directly with real values and the operators are adapted accordingly.

Mutation.

- **Gaussian (normal) mutation:** $x_i' = x_i + \sigma_i \cdot N(0,1)$. The parameter σ (the step width) is crucial; it may be fixed, deterministically decreasing (e.g. $\sigma(t) = 1 - 0.9\,t/T$), adaptive (the 1/5 rule) or self-adaptive (part of the genome, see chapter 3).
- **Cauchy mutation:** heavier tails ⇒ more frequent large jumps ⇒ better escape from local optima (used in *fast EP*).
- **Uniform mutation:** x_i' drawn at random from $[a_i, b_i]$ — unbiased.
- Handling of the bounds: clipping, reflection, periodic wrapping or resampling.

Crossover. Two types are distinguished:

- **Structural (discrete):** 1-point, uniform — the values are merely rearranged between the parents; the offspring is a vertex of the hyperbox given by the parents.
- **Arithmetic:** the values are combined.

Arithmetic crossover

For parents $\vec{x}, \vec{y}$ and $\alpha \in (0,1)$:

$$\vec{z} \;=\; \alpha\vec{x} + (1-\alpha)\vec{y} \qquad \text{(convex combination)}.$$

Variants: *whole* arithmetic crossover (all components are crossed — the most common one), *simple arithmetic* (only one randomly chosen component), *single* (a combination with one-point crossover: beyond the cut point the values are combined arithmetically). For $\alpha = 1/2$ it is the plain average of the parents.

Example. $\vec{x} = (2.0;\ -1.0;\ 4.0)$, $\vec{y} = (0.0;\ 3.0;\ 1.0)$, $\alpha = 0.25$:

$$\vec{z} = 0.25\,\vec{x} + 0.75\,\vec{y} = (0.5;\ 2.0;\ 1.75).$$

The disadvantage of purely arithmetic crossover: the offspring lie inside the convex hull of the parents, so the variance of the population necessarily decreases ("shrinking"). Extended variants are therefore used:

- **BLX-α** (*blend crossover*): z_i is drawn uniformly from the interval $[\min - \alpha I,\ \max + \alpha I]$, where $I = |x_i - y_i|$; typically $\alpha = 0.5$. It allows the offspring to lie outside the parents as well.
- **SBX** (*simulated binary crossover*): it imitates the behaviour of one-point binary crossover in a real-valued space; the offspring are distributed symmetrically around the parents with a density controlled by the parameter η. The standard operator in NSGA-II.

1.3.3 Permutation representation

An individual is a permutation π of $\{1, \ldots, n\}$. Typical problems: the travelling salesman problem (TSP), scheduling, assignment problems, ordering of operations. The classical operators fail here — the result of one-point crossover of two permutations is in general not a permutation. Operators are therefore needed that *preserve feasibility* and at the same time inherit something meaningful. Which information should be inherited depends on the problem:

operator	what it preserves	typical use
PMX	absolute positions of the cities	problems where position matters
OX	relative order	TSP (cyclic tours)
CX	absolute positions (cycles)	problems where position matters
ER	edges (adjacencies)	TSP — the best quality

PMX — partially mapped crossover

PMX (partially mapped crossover, Goldberg)

A two-point operator. (1) Choose the segment between the two cut points and exchange it between the parents. (2) From the exchanged segments derive the *mapping* of the corresponding values. (3) Copy the remaining positions from the original parent; where a conflict would arise (the value is already in the offspring), use the mapping (repeatedly if necessary).

Example. $P_1 = (123|4567|89)$, $P_2 = (452|1876|93)$.

1. Exchange of the segments: $O_1 = (\ldots|1876|\ldots)$, $O_2 = (\ldots|4567|\ldots)$.
2. Mapping from the segments: $1 \leftrightarrow 4$, $8 \leftrightarrow 5$, $7 \leftrightarrow 6$, $6 \leftrightarrow 7$.
3. Filling in the conflict-free positions from P_1 into O_1: $O_1 = (.23|1876|.9)$ (the values 1 and 8 would collide).
4. The conflicts are resolved by the mapping: $1 \to 4$, $8 \to 5$, hence $O_1 = (423|1876|59)$. Analogously $O_2 = (182|4567|93)$.

OX — order crossover

OX (order crossover, Davis)

A two-point operator preserving the *relative order*. (1) Copy the segment between the cut points from the first parent. (2) From the second parent read the values cyclically, starting at the position after the second cut point, and skip those that are already in the offspring. (3) Insert the remaining values into the free positions of the offspring, again cyclically from the position after the second cut point.

Example. $P_1 = (123|4567|89)$, $P_2 = (452|1876|93)$.

1. O_1 takes over the segment from P_1: $O_1 = (\cdots|4567|\cdot\cdot)$.
2. From P_2 we read cyclically from position 8: $9, 3, 4, 5, 2, 1, 8, 7, 6$.
3. We delete the values already contained $(4, 5, 6, 7)$: $9, 3, 2, 1, 8$ remain.
4. We fill in from position 8 cyclically (positions $8, 9, 1, 2, 3$): $O_1 = (218|4567|93)$.
5. Symmetrically (segment from P_2, order from P_1): $O_2 = (345|1876|92)$.

The principle: the inner segment is taken over unchanged, the rest is filled in in the relative order of the second parent, skipping duplicates. For the TSP, OX is more suitable than PMX, because in

a round tour it is not the absolute position of a city that matters, but the order.

CX — cyclic crossover

CX (cyclic crossover, Oliver)

It preserves the *absolute position*: every gene of the offspring comes from the same position in one of the parents. Cycles are constructed: start at position 1 and take the value from P_1; look at which value stands at the same position in P_2, find it in P_1 and continue until the cycle closes. Take the even cycles from one parent and the odd ones from the other.

Example. $P_1 = (123456789)$, $P_2 = (412876935)$. We start with the value 1 from P_1 at position 1. Below it, in P_2, is the value 4, which we find in P_1 at position 4 $\Rightarrow$ we add it; below it is 8, which is in P_1 at position 8 $\Rightarrow$ we add it; below it is 3 (position 3), below that 2 (position 2), and below 2 is 1 — the cycle has closed. We have $O_1 = (1\,2\,3\,4\,\_\,\_\,\_\,8\,\_)$; the rest is filled in from P_2: $O_1 = (123476985)$, and analogously $O_2 = (412856739)$.

ER — edge recombination An observation: PMX, OX and CX preserve only about 60 % of the parents' edges. For the TSP, however, it is precisely the *edge* that carries the quality. *Edge recombination* (Whitley) therefore:

1. For every city it builds a *neighbour list* from **both** parents.
2. It starts at a random (or the first) city.
3. The next city is the not-yet-visited neighbour that has the *fewest* remaining neighbours (the heuristic being: do not leave cities "stranded"); ties are broken at random.
4. If the neighbour list turns out to be empty, one continues with a random unvisited city (this happens in 1–1.5 % of the cases).

ER2 additionally marks the edges that occur in *both* parents (with the symbol #) and prefers them when choosing.

Example. $P_1 = (123456789)$, $P_2 = (412876935)$ (cyclic tours). Neighbour lists: 1: 9, 2, 4; 2: 1, 3, 8; 3: 2, 4, 9, 5; 4: 3, 5, 1; 5: 4, 6, 3; 6: 5, 7, 9; 7: 6, 8; 8: 7, 9, 2; 9: 8, 1, 6, 3. Start at 1: the candidates are 9 (4 neighbours), 2 (3), 4 (3) — 9 drops out, between 2 and 4 we draw lots and 4 comes out. The neighbours of 4 are 3 and 5; 3 still has 3 free neighbours, 5 only 2, so we take 5, and so on — the result is e.g. (145678239).

Technical note: strictly speaking, the visited city is first removed from *all* the lists and only then are the remaining neighbours counted (above, at the start, the counts are given *before* removing the 1: after the removal they would be 9:3, 2:2, 4:2). Here this changes nothing about the order of the candidates, but when computing on paper one has to proceed this way, otherwise different ties come out.

Mutation of permutations

- **Swap:** exchange of two random cities.
- **Insert:** removal of a city and its insertion elsewhere (the rest is shifted).
- **Inversion (a 2-opt step):** reversal of a contiguous segment — the most effective one for the TSP, because it changes only two edges.
- **Scramble:** random shuffling of a contiguous segment.
- **Exchange of subtours** and heuristic mutations (2-opt, 3-opt, Lin-Kernighan).

1.3.4 Other representations

Trees (section 1.4), graphs and DAGs (Cartesian GP, neuroevolution), matrices (schedules, the TSP as a predecessor matrix), finite automata (EP), rule lists (LCS, section 6.1), artificial-life agents.

1.4 Genetic and evolutionary programming

1.4.1 History

The idea of evolving programs was already mentioned by Alan Turing in the 1950s. Forsyth (1980, the BEAGLE system) applied it to pattern recognition, Nichael Cramer (1985) was the first to describe tree-shaped individuals, and John Koza (1989–1992) formulated tree-based genetic programming in its present form (including a patent).

Genetic programming (GP)

An evolutionary algorithm whose individuals are **executable structures** — programs, expressions, circuits, models. Fitness is determined by *running* the individual on test data. Neither the size nor the shape of an individual is fixed; both change in the course of evolution.

Algorithm 3 The general scheme of GP

1: generate an initial population of random programs
2: **while** a sufficiently good solution has not been found **do**
3: evaluate the programs by **running** them on the training data
4: selection according to fitness (typically a tournament)
5: crossover of two programs, mutation of programs
6: create the new population
7: **end while**

1.4.2 Tree-based GP

Representation. A program is a *syntax tree*. We define two sets:

- **The terminal set** T — the leaves: variables, constants, functions without arguments (e.g. reading a sensor).
- **The non-terminal (function) set** F — the inner nodes with a given *arity*: arithmetic operations, logical connectives, conditionals, loops.

Two properties are required: *closure* — every function must be able to process any output of any other one (hence the "protected division" %, which returns 1 for a zero divisor) — and *sufficiency* — the solution must be expressible from $T \cup F$ at all.

Example: the tree `(+ (* x x) (sin y))` represents the expression $x^2 + \sin y$.

Initialisation.

- **Grow:** the nodes are drawn from $T \cup F$ until the limit on depth / number of nodes is reached ⇒ irregular trees of various sizes.
- **Full:** up to the given depth only F is drawn from, at the last level only T ⇒ full, regular trees.
- **Ramped half-and-half** (Koza): half of the population by the grow method, half by the full method, for uniformly distributed depths (e.g. 2–6). The standard choice; it produces rather "bushy" regular shapes, which suits problems with symmetries where all the inputs are similarly important.
- **Uniform sampling** of all possible trees gives more irregular shapes (some terminals closer to the root).
- **Seeding:** inserting partial or expert solutions into the initial population — it speeds things up, but premature convergence is a risk.

Crossover.

- **Subtree exchange** (Koza's standard crossover): a random node is chosen in each parent and the subtrees below them are exchanged. Non-terminals tend to have a higher probability of being

chosen (typically 90 % non-terminal, 10 % terminal) — otherwise only the leaves would often be exchanged.

- **Homologous crossover** tries to preserve the position of the genetic material. First the *common region* C is found (by traversing from the root as long as the arities of the nodes agree):
  - *One-point:* the crossover point is chosen from the common region.
  - *Uniform (GPUX, Poli and Langdon):* every node of the common region is exchanged with a constant probability; nodes *inside* C are exchanged without any effect on their subtrees, nodes on the *boundary* of C are exchanged together with their subtrees. In the early phases large subtrees are thus exchanged, and as the population converges the operator becomes ever more local.
- **Context preserving crossover**: a node is identified by its *coordinates* — the sequence of branchings on the path from the root (e.g. $(2, 1, 3, 1)$). The *strong* variant allows crossover only between nodes with the *same* coordinates, the *weak* one is more permissive.

Mutation. In GP it is necessary (not merely useful) to use *several types* of mutation:

- **Subtree mutation** — replaces a random subtree by a new random one.
- **Size-fair** subtree mutation — the size of the new subtree is drawn so as to match the removed one.
- **Node mutation** (node replacement) — a node is exchanged for another one of the *same arity*.
- **Hoist** — the individual is replaced by one of its subtrees (it shrinks).
- **Shrink** — a subtree is replaced by a terminal (it shrinks).
- **Permutation mutation** — the subtrees of one non-terminal are swapped.
- **Mutation of constants** — GP is traditionally bad at fine-tuning numerical values. Arithmetic mutation of constants helps, or even local optimisation of *all* the constants in the tree (hill climbing, a gradient method) — a memetic approach.

Fitness and selection. Fitness = the error on the training data (symbolic regression), the number of correct answers, the quality of the behaviour in a simulation. Selection is usually by tournament. The risk of overfitting is the same as in machine learning (it is handled by a validation set and by penalising complexity).

1.4.3 Bloat

Bloat

Bloat is the tendency of programs in GP to grow in size without a corresponding improvement in fitness. Typically it is the accumulation of *introns* — pieces of code that have no effect on the output (e.g. `(+ x 0)`, `(if false ...)`).

Why it happens: in the space of programs there are far more large programs with the same behaviour than small ones, so neutral drift alone pushes towards growth; moreover, introns protect an individual against destructive crossover, so they "hitchhike" along with the good code. In nature growth runs into physical and energetic limits; in GP bloat has to be fought explicitly:

- **Hard limits** on the depth or the number of nodes (an offspring exceeding the limit is discarded or replaced by the parent).
- **A penalty in the fitness** (*parsimony pressure*): $f' = f - \alpha \cdot \text{size}$; or a *lexicographic* variant: with equal fitness the smaller individual wins.
- **Anti-bloat operators** — watch crossover and mutation so that they do not enlarge the individual much (size-fair variants), plus the shrinking mutations *hoist* and *shrink*.
- **A multi-objective approach** — size as a second criterion (section 6.4).

1.4.4 ADFs and typed GP

ADF (automatically defined functions)

Subprograms evolved together with the main program — a necessary condition for **modularity**, higher complexity and the ability to capture the symmetries of the problem. An individual consists of a `main` branch and one or more ADF branches; every ADF is characterised by its *arity* and may have a restricted (or different) set of terminals and non-terminals. A call of an ADF behaves in the main program as a **new non-terminal** of the given arity. The genetic operators work on the branches **separately** (main is crossed with main, ADF_1 with ADF_1). Extensions: automatically defined loops, iterations and recursion. An alternative is the *coevolution* of two populations — of main programs and of subprograms (section 3.3).

Enforcing structure. *Strongly typed GP* gives every node a type of its inputs and output and the operators may create only type-correct trees; *grammar-based GP* describes the admissible shapes of the trees by a context-free grammar. Both restrict the search space to meaningful programs.

1.4.5 Linear and graph GP

Linear GP. A program is a sequence of instructions (often machine code or byte code) operating on registers. Advantages: simple operators (vector crossover and mutation), fast interpretation, natural closeness to real hardware. Disadvantage: a high risk of producing nonsensical programs. It is popular in artificial life and in the evolution of game bots. A famous example: **Tierra** (T. Ray) — a simulation of an artificial ecosystem in which the individuals are self-copying machine-code programs competing for memory and processor time; parasites, hyperparasites and defences against them emerged. Successors: Avida, Avida-ED.

Graph GP. A program is a general (often acyclic) graph. Originally an extension of tree GP to parallel programs; it turned out that graphs describe electrical circuits, finite automata, neural networks and plans well. The problem is the complexity of the operators — how to cross general graphs.

Cartesian GP (CGP). [♦ beyond the slides] An elegant way around this: a DAG is represented by its planar image in a 2D grid with the coordinates of the nodes (the phenotype), whereas the genotype is a **linear** sequence of integers describing, for each node, its function (an index into a table of functions), its inputs, and finally the output nodes of the graph. Properties:

- Mutation is a point mutation of the linear genome; one has to make sure that only valid functions and valid connections arise (a limit on the depth of the links).
- A small change of the genotype may have a large impact on the phenotype; conversely, a number of mutations are *neutral* (nodes not connected to the output form natural introns, which is beneficial for CGP).
- Crossover is of little importance; a naive one-point crossover is too destructive, so an arithmetic variant and cuts at node boundaries are used.
- Selection: usually a $(1 + \lambda)$-ES with $\lambda = 4$.
- The genome has a constant length (bloat is limited by the grid).

Developmental GP. Instead of evolving a graph directly, one evolves a *tree-program that builds the graph step by step.* One starts with an "embryo" and applies operations for the structure (serial/parallel connection, three-way splitting) and for the elements (insert a resistor, a capacitor). Applications: Koza — the evolution of electrical circuits (the rediscovery of patented designs), Gruau — *cellular encoding* for neural networks, AutoML — the evolution of scikit-learn models.

1.4.6 Grammatical evolution

Grammatical evolution (GE)

The genotype is a **linear list of integers** (of variable length), the phenotype is a program derived from a context-free grammar. A number from the genome determines which rule is used to expand the leftmost non-terminal:

$$\text{rule} \;=\; (\text{gene mod the number of rules for this non-terminal}).$$

The use of the modulo makes the encoding **robust** (every genome is interpretable) and, because the genome is linear, **the standard GA operators** can be used — that is the main practical advantage. The grammar moreover guarantees syntactic correctness. The drawback is non-locality: a small change of the genotype (especially at the beginning) may change the phenotype completely (the *ripple effect*). If the derivation does not terminate, the genome is read again from the beginning (*wrapping*) with a limit on the number of wraps.

1.4.7 Evolutionary programming

Evolutionary programming (EP)

A branch of EAs (L. Fogel, 1965 — older than GAs) whose goal was to evolve "artificial intelligence": an agent that predicts the state of the environment and chooses an appropriate action. The agent is represented by a **finite automaton**; **genotype = phenotype** (no special encoding). **Crossover is not used**, all variability comes from mutations.

EP over automata. The environment is a sequence of symbols of a finite alphabet. The automaton receives the symbols on its input, its output is compared with the next symbol of the sequence; fitness = the success of the prediction (absolute error, mean squared error), often with a penalty for the size of the automaton. Mutations: a change of an output symbol, a change of the target state of a transition, the addition of a state, the removal of a state, a change of the initial state. Half of the population is replaced by new automata.

A famous example — predicting primes. The goal is to predict the sequence

$$11101010001010001010001000001 0\ldots,$$

where 1 means "this number is a prime" (i.e. to learn the sieve of Eratosthenes). One proceeds iteratively: first a short sequence is learned, then it is extended by one symbol. Fitness: a point for every correctly guessed symbol minus $0.01 \cdot N$ for the number of states. The result is instructive: the automata kept simplifying until a one-state automaton generating a constant 0 emerged — primes are sparse, after all, so the constant answer "not a prime" has a high success rate. A classic illustration of how evolution exploits a badly designed fitness function.

EP today.

- The encoding should be *natural* for the problem (usually genotype = phenotype).
- A set of "clever", domain-specific mutations; no crossover.
- Parent selection: every individual is a parent exactly once.
- Environmental selection: a tournament between parents and offspring.
- EP over real-valued vectors contains — just as ES does — the mutation variances in the genome ($\sigma'_j = \sigma_j(1 + \alpha N(0,1))$, then $x'_j = x_j + \sigma'_j N(0,1)$). EP and ES have thus practically merged.

1.4.8 Is crossover good for anything? The headless chicken experiment

The traditional defence of crossover = the exchange of building blocks. The criticism (typical of EP): crossover is just a strange large random mutation which, moreover, does not respect the encoding.

Headless chicken test (Jones, 1995)

We compare standard crossover with *random* crossover, in which an individual is crossed with a **randomly generated** string. If the result is the same or better, then the operator under study does not in fact work as crossover but as a macromutation — "let us not call a headless chicken a chicken, even though it has many chicken-like features".

Interpretation: if clear building blocks exist, true crossover is better. If random crossover wins, it means that in the given problem/encoding there are no building blocks — a bad encoding, a bad operator, or a problem unsuitable for crossover.

1.5 Exam summary

- EA = population-based stochastic search; the cycle *initialisation* → *evaluation* → *parent selection* → *recombination* → *mutation* → *evaluation* → *environmental selection*.
- Variation (crossover, mutation) creates diversity, selection removes it and steers the search; designing an EA = finding the exploration/exploitation balance.
- Selection: **roulette wheel** ($p_i = f_i / \sum f_j$, high variance, sensitive to scaling), **SUS** (same expected values, low variance), **tournament** (pressure controlled by k, invariant to fitness transformations, needs no explicit fitness), **rank-based** (constant pressure), **Boltzmann** (pressure grows with time).
- Elitism guarantees monotonicity of the best fitness; the generational and steady-state models differ in the proportion of the population replaced.
- No Free Lunch: there is no universally best algorithm ⇒ the representation and the operators must be specialised to the domain.
- The historical branches GA / ES / EP / GP and their characteristic choices (see the comparison table) are worth being able to list from memory.
- SGA = binary strings + roulette wheel + 1-point crossover + bit-flip mutation + generational replacement. Parameters: n, $p_C \approx 0.7$, $p_M \approx 1/m$.
- Binary encoding of real numbers: $x = a + z(b-a)/(2^k - 1)$; the problem of Hamming cliffs is solved by the Gray code.
- Crossover: 1-point (disrupts a schema with probability $d(S)/(m-1)$), 2-point (no end bias), uniform (most disruptive, no positional bias).
- Mutation is secondary in a GA — it prevents the irreversible loss of alleles; inversion is a historical operator for evolving the ordering of genes.
- Fitness scaling (linear, sigma truncation, power, rank, Boltzmann) repairs the weaknesses of fitness-proportionate selection: the dominance of supermen at the beginning and the loss of pressure at the end.
- Representation + operators = fitness landscape. For nominal attributes only unbiased mutations, for ordinal ones biased (creep) mutations as well.
- Real-valued representation: Gaussian mutation $x' = x + \sigma N(0,1)$; arithmetic crossover $\alpha\vec{x} + (1-\alpha)\vec{y}$ (it reduces the variance), hence BLX-α and SBX.
- Permutations: PMX (positions + mapping), OX (relative order), CX (absolute positions by means of cycles), ER (edges, the best for the TSP). Be able to demonstrate PMX and OX on a concrete eight-element example!
- Mutation of permutations: swap, insert, inversion (2-opt), scramble.
- Tree GP (Koza): terminals + non-terminals, closure and sufficiency; initialisation by grow / full / ramped half-and-half; crossover = subtree exchange (non-terminals with a higher probability), several types of mutation including mutation of constants; fitness = running the program.

- Bloat = growth in size without an improvement in fitness (introns); three explanations: replication accuracy, removal bias, crossover bias. Defences: size limits, penalties (parsimony pressure), anti-bloat operators (hoist, shrink, size-fair), a multi-objective approach.
- ADFs = evolved subprograms (modularity); the operators work on the branches separately. Typed GP and grammar-based GP restrict the space to meaningful programs.
- Variants: linear GP (byte code, Tierra), graph GP, Cartesian GP (a linear genotype → a 2D DAG, point mutation, a $(1+4)$-ES), developmental GP (a tree builds a graph — circuits, networks).
- Grammatical evolution: a linear genome of integers, the choice of a grammar rule by means of the modulo, a derivation tree → the phenotype; a robust encoding and the standard GA operators, but a non-local mapping.
- EP: finite automata, genotype = phenotype, mutation only, tournament selection from parents and offspring together; the example of predicting primes (a degenerate fitness).
- The headless chicken experiment: a test of whether crossover really combines building blocks or is merely a macromutation.

2 Schema theory and probabilistic models of the SGA

The chapter answers the question of *why* genetic algorithms work. First Holland's classical schema theory (and its criticism, which matters just as much at the examination as the theorem itself), then the exact Vose model that superseded it.

2.1 Schema theory

Schema theory is the first attempt to explain formally *why* genetic algorithms work. It comes from Holland (1975) and was popularised by Goldberg (1989). Today it is regarded as historically important but insufficient; nevertheless it is a standard part of the examination, and its criticism is just as important as the theorem itself.

2.1.1 A schema, its order and defining length

Schema

Let an individual be a word of length m over the alphabet $\{0,1\}$. A **schema** S is a word of length m over the alphabet $\{0,1,*\}$, where $*$ means "don't care". A schema represents the set of all individuals that agree with it in all fixed positions; a schema with r stars represents 2^r individuals.

Example: the schema $1*0*$ represents $\{1000, 1001, 1100, 1101\}$; the schema $**\ldots*$ represents the whole space.

Order, defining length, fitness of a schema

For a schema S of length m we define:

- **Order** $o(S)$ = the number of fixed positions (the number of zeros and ones).
- **Defining length** $d(S)$ = the distance between the first and the last fixed position ($d(S) = 0$ for a schema with a single fixed position).
- **Fitness of the schema** $F(S,t)$ = the average fitness of those individuals in the population $P(t)$ that represent the schema S. *Note:* this value depends on the current population, not only on S and f.

Example. For $S = 1{*}{*}0*1$ ($m = 6$) we have $o(S) = 3$ (positions 1, 4, 6) and $d(S) = 6 - 1 = 5$.

Combinatorics of schemata.

- There are 3^m schemata of length m in total.

- A single individual of length m is a representative of 2^m schemata (at each position either its own bit or $*$).
- A population of n individuals represents between 2^m and $n \cdot 2^m$ schemata (depending on how similar they are).

2.1.2 Holland's schema theorem

Theorem 2.1 (Schema theorem (TST), Holland 1975)**.** *In a simple genetic algorithm with fitness-proportionate selection, one-point crossover with probability p_C and bit mutation with probability p_M, the expected number of representatives of a schema S in the next generation satisfies*

$$\mathbb{E}\big[C(S,t+1)\big] \;\geq\; C(S,t)\,\frac{F(S,t)}{\bar{f}(t)}\left[1 - p_C\frac{d(S)}{m-1} - p_M\,o(S)\right].$$

In words: ***short*** *(small defining length),* ***low-order*** *and* ***above-average*** *schemata multiply exponentially over successive generations.*

Derivation Let $C(S,t)$ denote the number of individuals in $P(t)$ representing S, let $n = |P(t)|$ and let $\bar{f}(t)$ be the average fitness of the population. We proceed in three steps, one per operator.

Step 1 — selection. The probability of selecting an individual v by the roulette wheel is $p_s(v) = f(v)/F(t)$, where $F(t) = \sum_{u\in P(t)} f(u)$. The probability that the selected individual represents S is

$$p_s(S) \;=\; \sum_{v\in S\cap P(t)} \frac{f(v)}{F(t)} \;=\; \frac{C(S,t)\,F(S,t)}{F(t)}.$$

For n independent draws we therefore have

$$\mathbb{E}\big[C(S,t+1)\big] \;=\; n\,p_s(S) \;=\; C(S,t)\,\frac{F(S,t)}{\bar{f}(t)}, \qquad \bar{f}(t) = F(t)/n.$$

If the schema is above average by ε, i.e. $F(S,t) = \bar{f}(t)\,(1+\varepsilon)$, and this ratio is maintained in time, we get

$$C(S,t+1) = C(S,t)(1+\varepsilon) \qquad\Longrightarrow\qquad C(S,t) = C(S,0)\,(1+\varepsilon)^t,$$

that is, **exponential growth** (or decay for $\varepsilon < 0$).

Step 2 — crossover. One-point crossover chooses the cut point from $m-1$ possibilities. The schema is *certainly* disrupted if the cut falls between its first and its last fixed position, i.e. with probability at most $d(S)/(m-1)$ (at most, because even with a cut inside, the schema may be restored if the other parent contains the same values). Crossover is performed with probability p_C, so

$$p_{\text{surv}}^{\text{cross}}(S) \;\geq\; 1 - p_C\,\frac{d(S)}{m-1}.$$

Step 3 — mutation. The schema survives if none of its $o(S)$ fixed positions is changed:

$$p_{\text{surv}}^{\text{mut}}(S) = (1-p_M)^{o(S)} \;\approx\; 1 - p_M\,o(S) \quad \text{for } p_M \ll 1.$$

Putting it together. Assuming (approximate) independence of the two destructive events and neglecting their product:

$$\mathbb{E}\big[C(S,t+1)\big] \;\geq\; C(S,t)\,\frac{F(S,t)}{\bar{f}(t)}\left[1 - p_C\frac{d(S)}{m-1} - p_M o(S)\right]. \qquad \blacksquare$$

Numerical example Take Goldberg's population from p. 5 ($m = 5$, $n = 4$): 01101 (169), 11000 (576), 01000 (64), 10011 (361); $\bar{f} = 292.5$. The parameters are $p_C = 0.7$, $p_M = 0.001$.

schema S	representatives	$F(S)$	$o(S)$, $d(S)$	survival	$\mathbb{E}[C(S, t+1)]$
1∗∗∗∗	11000, 10011	468.5	1; 0	$1 - 0 - 0.001 = 0.999$	$2 \cdot 1.602 \cdot 0.999 = 3.2$
1∗∗∗1	10011	361	2; 4	$1 - 0.7 - 0.002 = 0.298$	$1 \cdot 1.234 \cdot 0.298 = 0.37$
0∗∗∗∗	01101, 01000	116.5	1; 0	0.999	$2 \cdot 0.398 \cdot 0.999 = 0.80$

Interpretation: the short above-average schema 1∗∗∗∗ spreads from 2 to about 3.2 representatives. The schema 1∗∗∗1 is above average as well ($F/\bar{f} = 1.23$), but its large defining length is fatal — crossover disrupts it and we expect it to disappear. The below-average 0∗∗∗∗ declines. This is exactly what the theorem says.

2.1.3 The building blocks hypothesis

Building blocks hypothesis (BBH)

A genetic algorithm searches for a good solution by **recombining short, low-order and above-average schemata** — the so-called *building blocks* — into ever better solutions. Goldberg's metaphor: "just as a child builds a castle out of simple wooden blocks, so a GA seeks near-optimal solutions by putting building blocks together".

Practical consequences:

- **The encoding matters.** Genes that interact with each other should lie close together (*linkage*), otherwise crossover disrupts them. Hence the attempts at inversion, *messy GAs*, *linkage learning* and estimation of distribution algorithms (EDA).
- **The population size matters.** The population must contain enough samples of every building block (*population sizing* theory).
- **Premature convergence is harmful** — the loss of diversity makes it impossible to discover the missing blocks.

2.1.4 Implicit parallelism and the multi-armed bandit

Implicit parallelism

A GA works explicitly with n individuals, but implicitly it "processes" many more schemata at the same time — between 2^m and $n \cdot 2^m$. Holland estimated that the number of schemata that are processed *usefully* (they grow exponentially and are not disrupted too much) is of the order of n^3.

This is often commented on as the only case in which the combinatorial explosion works in our favour.

Connection with the bandit problem. Holland's original motivation was an "adaptive plan" seeking a balance between exploration and exploitation. Formally:

- **The two-armed bandit.** We have N coins and a machine with two arms with unknown mean payoffs μ_1, μ_2 and variances. We allocate n coins to the worse arm and $N - n$ to the better one. The analytical solution says that the optimal strategy allocates *exponentially* more trials to the currently winning arm: $N - n^* = O(e^{c\,n^*})$.
- **The GA as a bandit.** The schema theorem says that a GA allocates exponentially more "slots in the population" to the more successful schemata — that is, that it solves the exploration/exploitation dilemma (almost) optimally.

- It was originally believed that the SGA plays a single game with 3^m arms. In fact it plays many games in parallel: the schemata compete for concrete fixed positions, and schemata of order k play a game with 2^k arms.

2.1.5 Criticism of schema theory

When a GA fails — deceptive problems Consider a function in which the schemata

$$S_1 = (111*\ldots*), \qquad S_2 = (*\ldots*11)$$

are above average, but at the same time

$$f(111*****11) \ll f(000*****00).$$

Selection leads the GA towards the blocks of ones, even though the optimum lies elsewhere. Such problems are called **deceptive** (Goldberg 1987): the partial building blocks do not lead to the global optimum when put together. Deception is sometimes proposed as a measure of the difficulty of a problem for an EA (not everyone agrees with this).

The opposite test are the **Royal Road functions** (Mitchell, Forrest, Holland): fitness is the sum of contributions for complete "blocks" of ones (e.g. eight blocks of eight bits each), so the landscape is tailored exactly to the building blocks hypothesis and a GA ought to excel on it. The experiment turned out the other way round: the simple GA lost to a random mutation hill climber (RMHC). The cause is *hitchhiking*: together with a good block that has been found, selection also spreads incidental "parasitic" bits at the other positions, and these destroy the diversity needed to find the remaining blocks. The Royal Road functions are therefore the standard empirical counterexample to a naive reading of the BBH.

Static average fitness — the static BBH Grefenstette (1991): people interpret the BBH as saying that a GA converges to solutions with a good *true static* average schema fitness (over the whole space). A GA, however, estimates the fitness of a schema only *from the samples in the population*, and that estimate is biased for two reasons:

- **Collateral convergence.** As soon as a GA converges somewhere, it stops sampling the schemata uniformly. If, for instance, the schema $111*\ldots*$ is good, then after a few generations almost every individual has this prefix. But then almost every sample of the schema $***000\ldots$ is at the same time a sample of $111000\ldots$, and the fitness of the first schema is estimated in a completely distorted way.
- **Large variance of schema fitness.** Consider $f(x) = 2$ for $x \in 111*\ldots*$, $f(x) = 1$ for $x \in 0*\ldots*$ and 0 otherwise. The static average fitness of the schema $1*\ldots*$ is $\approx 1/2$, whereas $F(0*\ldots*) = 1$. The GA, however, will estimate $F(1*\ldots*) \approx 2$, because individuals with the prefix 111 will prevail in the population. A high variance leads to a systematic bias and to the GA not sampling the schemata independently.

Summary of the weaknesses

- The theorem is an **inequality** and only a **one-step** result. Iterating it over several generations requires the assumption that $F(S,t)/\bar{f}(t)$ remains constant — which typically does not hold (both change dynamically).
- Populations are **finite and small** ⇒ sampling errors; the exponential growth is only an expected value.
- The theorem accounts only for the **destructive** effect of the operators, not for their *constructive* role (crossover can also create a schema).
- The competition of schemata is **strongly dependent**, not independent as the arms of a bandit are.

- The analysis is "on-line": the SGA maximises the running performance, so it is more suitable for adaptive problems, whereas the most common use is to find one best solution (an off-line criterion).

Even so, the TST remains a useful intuition: *what is short, simple and good will spread.*

2.2 Probabilistic models of the simple GA

In the 1990s (Vose, Liepins, Nix, Whitley) *exact* models of the SGA were developed, which replaced the approximate schema theory. The goal was to describe precisely what a population looks like, to describe the transition mapping between populations, and to study its properties and the asymptotic behaviour of the SGA. Two approaches are distinguished: **infinite populations** (a deterministic dynamical system) and **finite populations** (a Markov chain).

2.2.1 The simplified SGA

For the analysis the SGA is simplified even further: in each step two parents are selected, with probability p_C they are crossed, **one of the offspring is discarded**, and the remaining one is mutated and inserted into the new population. In this way the creation of each individual of the new population is independent.

2.2.2 Formalisation of the population

We identify every binary string of length l with a number $i \in \{0, 1, \ldots, 2^l - 1\}$ (e.g. $00000111 \equiv 7$). The population at time t is described by two vectors of length 2^l:

Population vector and selection vector

- $p(t) = (p_0(t), \ldots, p_{2^l-1}(t))$, where $p_i(t)$ is the *relative frequency* of the string i in the population. We have $p_i \geq 0$, $\sum_i p_i = 1$, so $p(t)$ lies in the simplex Λ.
- $s(t) = (s_0(t), \ldots, s_{2^l-1}(t))$, where $s_i(t)$ is the probability that selection picks the string i.

The vector p describes the *composition* of the population, the vector s the *selection probabilities.*

2.2.3 The operator $\mathcal{G}$

The goal is to find an operator $\mathcal{G} : \Lambda \to \Lambda$ such that

$$p(t+1) = \mathcal{G}\big(p(t)\big),$$

and to decompose it into two parts: $\mathcal{G} = \mathcal{M} \circ \mathcal{F}$, where $\mathcal{F}$ corresponds to selection (fitness) and $\mathcal{M}$ to the so-called *mixing* — crossover and mutation together. Then it suffices to iterate the operator from the initial population and we know the complete behaviour of the SGA.

The part $\mathcal{F}$ — selection. Define the diagonal matrix $\mathbf{F}$ with $\mathbf{F}_{ii} = f(i)$ and $\mathbf{F}_{ij} = 0$ for $i \neq j$. Fitness-proportionate selection is then exactly

$$s(t) \;=\; \frac{\mathbf{F}\, p(t)}{\sum_j \mathbf{F}_{jj}\, p_j(t)}.$$

The numerator $(\mathbf{F}p)_i = f(i)\, p_i$ is the "weight" of the string i in the population, the denominator is a normalising constant (the average fitness of the population). If we now drop crossover and mutation, the new population is simply n independent draws from the distribution $s(t)$, so $\mathbb{E}[p(t+1)] = s(t)$. Substituting into the definition of selection we obtain $\mathbb{E}[s(t+1)] \propto \mathbf{F}\, s(t)$, so iterating $\mathcal{F}$ is (up to normalisation) repeated multiplication by the matrix $\mathbf{F}$ — the *power method.* It converges to the eigenvector belonging to the largest diagonal value, i.e. to a population consisting exclusively of copies of the best individual *present in the initial population.* With finite populations, statistical

errors cause deviations from the expected value; the larger the population, the smaller the deviation. Infinite populations are exact, albeit impractical.

Mini-example. For $l = 2$ we have four strings $00, 01, 10, 11$, i.e. the indices $0, 1, 2, 3$. Let $f = (1, 2, 3, 4)$ and let a population of eight individuals contain 2×00, 4×01, 1×10, 1×11, i.e. $p = (0.25;\ 0.5;\ 0.125;\ 0.125)$. Then $\mathbf{F}p = (0.25;\ 1.0;\ 0.375;\ 0.5)$, the sum is 2.125 (= the average fitness of the population) and

$$s \;=\; (0.118;\ 0.471;\ 0.176;\ 0.235).$$

The share of the best string 11 has grown from 12.5 % to 23.5 %, the share of the worst one 00 has dropped from 25 % to 11.8 % — exactly by the factor $f(i)/\bar{f}$, as we know from the schema theorem.

The part $\mathcal{M}$ — mixing. We introduce the function

$$r(i, j, k) \;=\; \Pr[\text{offspring } k \text{ arises from parents } i, j],$$

which comprises both crossover (over all cut points) and mutation. Then

$$\mathbb{E}\big[p_k(t+1)\big] \;=\; \sum_i \sum_j s_i(t)\, s_j(t)\, r(i, j, k).$$

This is therefore a *quadratic* operator on the simplex. Deriving $r(i, j, k)$ is technically demanding but possible. The key simplification follows from the fact that both one-point crossover and bit mutation "do not see" the absolute values of the bits, only their agreements and disagreements. Formally: for any k we have

$$r(i, j, k) \;=\; r(i \oplus k,\ j \oplus k,\ 0),$$

where $\oplus$ is the bitwise XOR. It therefore suffices to know a single *mixing matrix* $\mathbf{M}$ with entries $\mathbf{M}_{ij} = r(i, j, 0)$ (of size $2^l \times 2^l$), and the rest is computed by the permutations $\sigma_k : x \mapsto x \oplus k$: $\mathcal{M}(s)_k = (\sigma_k s)^\top \mathbf{M}\, (\sigma_k s)$. This is the main technical contribution of Vose's formalism — the problem shrinks from 2^{3l} numbers to 2^{2l}.

2.2.4 Results for infinite populations

- The SGA is a **discrete dynamical system** on the simplex; the sequence $p(0), p(1), \ldots$ is a trajectory of this system.
- **Fixed points of $\mathcal{F}$**: populations formed exclusively by copies of the best individual from the initial population (selection "focuses").
- **Fixed point of $\mathcal{M}$**: a single one — the uniform distribution (mixing "defocuses", it maximises entropy).
- The fixed points of the composed $\mathcal{G}$ are not known in general; finding them is the main open problem. The behaviour is a combination of focusing and defocusing, which manifests itself as **punctuated equilibria** — long periods of metastability alternating with rapid transitions. This phenomenon is known from biology as well.

2.2.5 Finite populations: a Markov chain

Markov chain

A stochastic process in discrete time with states, where the probability of a transition to the next state depends *only* on the current state (not on the history). A complete description is given by the matrix of transition probabilities.

We model an SGA with a population of size n over strings of length l:

- **States** = the individual possible populations (multisets of n strings). Their number is

$$N = \binom{n + 2^l - 1}{2^l - 1},$$

  which is the number of multisets of size n from 2^l types.
- **The matrix Z**: the columns correspond to populations, $\mathbf{Z}(y, i)$ = the number of occurrences of the individual y in the population i. The columns of the matrix $\mathbf{Z}$ are thus the states of the chain.
- **The transition matrix Q** of size $N \times N$; it can be derived explicitly from $\mathcal{G}$ (the transition is a multinomial draw of n individuals from the distribution $\mathcal{G}(p)$):

$$\mathbf{Q}_{i,j} = n! \prod_{y=0}^{2^l-1} \frac{\left(\mathcal{G}(p_i)_y\right)^{\mathbf{Z}(y,j)}}{\mathbf{Z}(y,j)!}.$$

The size problem. Already for $n = l = 8$ the matrix $\mathbf{Q}$ has on the order of 10^{29} entries. The model is therefore exact but practically unusable for computation; its value is theoretical.

Qualitative consequences.

- If $p_M > 0$, the chain is **irreducible and aperiodic** (from any population any other can be reached by mutations) $\Rightarrow$ a unique stationary distribution exists and the SGA visits every population (including the one with the global optimum) infinitely many times with probability 1.
- Without elitism, however, it **does not converge** to the optimum — the optimum appears and disappears again. An *elitist* GA (one that keeps the best solution found) converges to the global optimum with probability 1 as $t \to \infty$. This is the standard convergence result; it says nothing, however, about the *speed*.
- There is a correspondence between the infinite- and the finite-population model: as $n \to \infty$ the behaviour of the finite model approaches the deterministic trajectory of $\mathcal{G}$.

2.2.6 Estimation of distribution algorithms (EDA)

[♦ beyond the slides: not in the EVA slides; it is the direct continuation of the Vose view]

EDA — the general scheme

Vose's model shows that the SGA is in fact a machine for transforming a probability distribution over the space of strings. *Estimation of distribution algorithms* take this literally: they throw away both crossover and mutation and maintain an **explicit model** $p_\theta(x)$. Iteration: sample a population from p_θ, select the better part, *re-learn* the model on it, repeat. The model θ is the only information carried between generations.
By the richness of the dependences captured: **none** (UMDA, PBIL), **pairwise** (MIMIC), **general** (BOA — a Bayesian network), **continuous** (CMA-ES). PBIL shifts the probability vector towards the best individual: $p_i \leftarrow (1 - \alpha)\, p_i + \alpha\, x_i^{\text{best}}$.

A univariate model fails as soon as there is *epistasis* between the genes — hence BOA, which learns the structure of the dependences. EDAs thus solve the problem for the sake of which Holland proposed inversion: where the cooperating genes should lie is something the algorithm learns by itself.

2.3 Exam summary

- Schema = a word over $\{0, 1, *\}$; $o(S)$ = the number of fixed positions; $d(S)$ = the distance between the first and the last fixed position; $F(S)$ = the average fitness of the representatives *in the given population*. There are 3^m schemata in total, and an individual is a representative of 2^m of them.

- TST: $\mathbb{E}[C(S,t+1)] \geq C(S,t)\frac{F(S)}{f}[1 - p_C\frac{d(S)}{m-1} - p_M o(S)]$; the derivation in three steps (selection $\rightarrow$ exponential growth, crossover $\rightarrow d(S)/(m-1)$, mutation $\rightarrow (1-p_M)^{o(S)}$).
- BBH: a GA assembles short, low-order, above-average schemata (building blocks). Consequences: the encoding matters, the population size matters, premature convergence is harmful.
- Implicit parallelism ($\sim n^3$ usefully processed schemata), the analogy with the k-armed bandit (exponential allocation of trials to the winner).
- Criticism: an inequality for a single step, finite populations, deceptive problems, collateral convergence, large variance of fitness, disregard of the constructive effects, dependence of the schemata.
- Vose's model: strings are identified with the numbers $0..2^l - 1$; the population is described by the vector of relative frequencies $p(t)$ in the simplex and by the vector of selection probabilities $s(t)$.
- We look for $\mathcal{G} = \mathcal{M} \circ \mathcal{F}$ with $p(t+1) = \mathcal{G}(p(t))$. $\mathcal{F}$ (selection) is given by the diagonal fitness matrix: $s = \mathbf{F}p / \sum_j \mathbf{F}_{jj} p_j$; $\mathcal{M}$ (crossover + mutation) is a quadratic operator $\mathbb{E}[p'_k] = \sum_{i,j} s_i s_j \, r(i,j,k)$. Thanks to the symmetry $r(i,j,k) = r(i \oplus k, j \oplus k, 0)$ a single mixing matrix suffices.
- Fixed points: $\mathcal{F}$ — a population of copies of the best individual; $\mathcal{M}$ — the uniform distribution. The combination leads to punctuated equilibria. The fixed points of $\mathcal{G}$ are not known.
- Finite populations: a Markov chain with states = populations, $N = \binom{n+2^l-1}{2^l-1}$ states; the matrix $\mathbf{Q}$ is exact but astronomically large. Irreducibility for $p_M > 0$; an elitist GA converges to the optimum with probability 1.
- Infinite populations = a deterministic model (exact, impractical), finite ones = a stochastic model (realistic, intractable). The limit transition connects them.
- **EDAs** take the view "a GA = a transformation of a distribution" literally: instead of operators a model p_θ is learned (UMDA/PBIL — independent marginals, MIMIC/BMDA — pairwise dependences, BOA — a Bayesian network, CMA-ES — a Gaussian). The cycle: sample $\rightarrow$ select the better ones $\rightarrow$ re-learn the model. PBIL: $p_i \leftarrow (1-\alpha)p_i + \alpha x_i^{\text{best}}$.

3 Evolution strategies, differential evolution, coevolution, open-ended evolution

The chapter joins four themes that have in common that they go *beyond* the classical genetic algorithm: two methods for continuous optimisation (ES and DE) and two themes in which the fitness itself changes — coevolution (fitness depends on the other individuals) and open-ended evolution (fitness is not fixed at all).

3.1 Evolution strategies

3.1.1 Motivation and basic features

Evolution strategies (ES) were proposed by Rechenberg and Schwefel in Berlin in the 1960s for the optimisation of real-valued parameters in difficult engineering problems (the shape of a nozzle, the bend of a pipe). Characteristics:

- An individual is a vector of real numbers; **mutation is the main operator**.
- Besides the *genetic parameters* (the solution itself) an individual also contains *strategy parameters*, also called *endogenous parameters*, which control the evolution itself — typically the standard deviations of the mutations. Hence the slogan "the evolution of evolution".
- Parent selection is **random** (independent of fitness); environmental selection is **deterministic** — the μ best are selected.
- The most modern and most successful representative is CMA-ES.

ES notation

μ — the population size (the number of parents), λ — the number of offspring created, ρ — the number of parents taking part in one offspring.

- $(\mu+\lambda)$-ES: the new population is selected from *both the μ parents and the λ offspring* (elitist).
- (μ,λ)-ES: the new population is selected *from the λ offspring only*; the parents always die out (non-elitist), necessarily $\lambda > \mu$.
- $(\mu/\rho \overset{+}{,} \lambda)$-ES: an explicit notation with the recombination of ρ parents.

	(μ,λ)	$(\mu+\lambda)$
pool for selection	offspring only	parents + offspring
elitism	no (fitness may decrease)	yes
escape from local optima	better	worse
convergence	slower	faster
recommendation	continuous domains, noisy or dynamic fitness	discrete domains, expensive fitness
self-adaptation of σ	works well (a bad σ dies out)	may get stuck (an old parent with a small σ survives)
ratio	typically $\lambda/\mu \approx 7$	even $\mu \geq \lambda$; $\lambda = 1$ = steady-state ES

3.1.2 $(1+1)$-ES and the 1/5 rule

The simplest ES: the population is a single individual, a single offspring is created by Gaussian mutation and is accepted only if it is better.

Algorithm 4 $(1+1)$-ES with the 1/5 rule (minimisation of $f : \mathbb{R}^n \to \mathbb{R}$)

initialise $\vec{x}(0)$ at random, $\sigma > 0$, $t \leftarrow 0$
repeat
  $\vec{y}(t) \leftarrow \vec{x}(t) + \sigma \cdot N(\vec{0}, \mathbf{I})$
  if $f(\vec{y}(t)) < f(\vec{x}(t))$ **then** $\vec{x}(t+1) \leftarrow \vec{y}(t)$
  else $\vec{x}(t+1) \leftarrow \vec{x}(t)$
  end if
  track p = the proportion of successful mutations over the last k iterations
  every k iterations adjust σ:
    if $p > 1/5$ **then** $\sigma \leftarrow \sigma / c$ ▷ going well: take larger steps, more exploration
    if $p < 1/5$ **then** $\sigma \leftarrow \sigma \cdot c$ ▷ going badly: shorten the steps, more exploitation
    if $p = 1/5$ **then** leave σ unchanged ▷ typically $0.8 \leq c \leq 1$
  $t \leftarrow t + 1$
until $\vec{x}(t)$ is good enough

The 1/5 success rule

A heuristic derived by Rechenberg from the analysis of two model functions (the sphere and the corridor): *the optimal rate of successful mutations is approximately* 1/5. If the success rate is higher, the step is too small (we are being too cautious) and σ is increased; if it is lower, the step is too large and σ is decreased. This is *adaptive* (not self-adaptive) parameter control — it uses feedback from the run, but the parameter is not part of the genome.

3.1.3 Self-adaptation of the strategy parameters

Self-adaptation

The strategy parameters (the deviations σ, possibly rotation angles) are **part of the genome** and are subject to mutation and selection together with the solution. The selection is indirect: individuals with a suitable σ produce better offspring, and thereby their σ spreads through the population. The order is essential: **first the strategy parameters are mutated, and only then are the genetic parameters mutated using them** — otherwise the old σ would be the one being evaluated.

An individual has the form $C = [\vec{x}, \vec{\sigma}]$, where $\vec{x} \in \mathbb{R}^n$ and $\vec{\sigma}$ has k components:

k	name	geometry of the mutation
1	one global σ	isotropic — the density contours are spheres
n	uncorrelated mutations	an ellipsoid with axes parallel to the coordinates
$n(n+1)/2$	correlated mutations	a general ellipsoid (n deviations + $n(n-1)/2$ rotation angles α_{ij}), corresponding to a full covariance matrix

The covariance matrix $\mathbf{C}$ is a product with a rotation matrix: $c_{ii} = \sigma_i^2$, $c_{ij} = \frac{1}{2}(\sigma_i^2 - \sigma_j^2)\tan(2\alpha_{ij})$.

The standard mutation rules (Schwefel).

$$\sigma_i' = \sigma_i \cdot \exp\big(\tau' N(0,1) + \tau N_i(0,1)\big), \qquad x_i' = x_i + \sigma_i' \cdot N_i(0,1),$$

where $\tau' \propto 1/\sqrt{2n}$ (a common perturbation for the whole individual) and $\tau \propto 1/\sqrt{2\sqrt{n}}$ (an individual perturbation of the component). The lognormal form guarantees the positivity of σ and the symmetry of increases and decreases. A lower bound $\sigma_{\min}$ is introduced so that σ does not degenerate to zero. The angles are mutated additively: $\alpha_{ij}' = \alpha_{ij} + \beta \cdot N(0,1)$.

3.1.4 Recombination in ES

One offspring is created from ρ parents. According to the number of parents we distinguish **local** ($\rho = 2$) and **global** ($\rho = \mu$) recombination; according to the way of combining:

- **Discrete (dominant):** the value at a given position is chosen at random from one of the parents.
- **Intermediate (arithmetic):** the value is the average of the parents' values at the given position.

The usual combination: discrete recombination for the genetic parameters (it preserves diversity) and intermediate recombination for the strategy parameters (it stabilises the adaptation of σ).

3.1.5 CMA-ES

[♦ beyond the slides: the slides give it one line as "today's most successful (and complex)" ES]

CMA-ES — the principle

Covariance Matrix Adaptation ES (Hansen, Ostermeier) is nowadays the reference algorithm for continuous black-box optimisation. Instead of self-adapting the individual σ values in the genome it maintains **a single distribution for the whole population**, $\mathcal{N}(\vec{m}, \sigma^2\mathbf{C})$, and learns its covariance matrix from the history of successful steps.
The iteration: sample λ points, select the μ best *by rank*, move the centre $\vec{m}$ to their weighted average, and update $\mathbf{C}$ and the step length σ according to the *evolution paths* — exponentially smoothed records of the direction in which the centre has been moving. If the steps go in a consistent direction, σ is increased; if they oppose each other, it is decreased.

Algorithm 5 The general ES cycle

$t \leftarrow 0$; initialise at random a population P_t of μ individuals $[\vec{x}, \vec{\sigma}]$
evaluate the individuals in P_t
while the termination condition is not met **do**
    $C_{t+1} \leftarrow \emptyset$
    for λ times **do**
        select ρ parents at random from P_t
        recombine them into a single offspring $[\vec{x}, \vec{\sigma}]$
        mutate the strategy parameters $\vec{\sigma}$
        mutate the genetic parameters $\vec{x}$ using the new $\vec{\sigma}$
        evaluate the offspring and insert it into C_{t+1}
    end for
    (μ, λ): $P_{t+1} \leftarrow$ the μ best of C_{t+1}
    $(\mu + \lambda)$: $P_{t+1} \leftarrow$ the μ best of $P_t \cup C_{t+1}$
    $t \leftarrow t + 1$
end while

The result is an algorithm that adapts automatically to the anisotropy and the correlations of the problem (in essence it learns an approximation of the inverse Hessian without a gradient), is invariant with respect to monotone transformations of fitness, and has very few tunable parameters.

3.2 Differential evolution

3.2.1 Motivation

Differential evolution (DE; Storn and Price, 1997) is a simple and surprisingly powerful algorithm for the black-box optimisation of functions $f : \mathbb{R}^D \to \mathbb{R}$. The key idea is **geometric**: neither the size nor the direction of the mutation step is estimated from a parameter σ; both are taken over from the *differences of pairs of individuals in the population*. The population thus encodes the scale and the orientation of the problem by itself: at the beginning it is scattered (large steps), in a converging state the differences are small (fine tuning). It is a kind of implicit self-adaptation for free.

The applications range from machine learning to the planning of optimal trajectories of space probes (the European Space Agency).

3.2.2 The DE/rand/1/bin algorithm

The population consists of N vectors $\vec{x}_1, \ldots, \vec{x}_N \in \mathbb{R}^D$. For **every** individual $\vec{x}_i$ (parent selection is thus trivial — every individual becomes a parent in turn) the following is performed:

1. **Mutation.** Select three mutually distinct individuals $\vec{x}_a, \vec{x}_b, \vec{x}_c$, all different from $\vec{x}_i$, and create the *donor vector*

$$\vec{v}_i \;=\; \vec{x}_a \;+\; F\,(\vec{x}_b - \vec{x}_c), \qquad F \in [0, 2],\ \text{typically } F \approx 0.8.$$

2. **Crossover (binomial, uniform "with a safeguard").** Create the *trial vector* $\vec{u}_i$:

$$u_{j,i} = \begin{cases} v_{j,i} & \text{if } \mathrm{rand}_j \le C \text{ or } j = I_{\mathrm{rand}}, \\ x_{j,i} & \text{otherwise,} \end{cases}$$

   where $\mathrm{rand}_j \sim U(0, 1)$, $C \in [0, 1]$ is the crossover threshold and I_{rand} is a random index from $\{1, \ldots, D\}$. The condition $j = I_{\mathrm{rand}}$ is the "safeguard": it guarantees that at least one component comes from the donor, so the offspring is never an identical copy of the parent.

3. **Environmental selection (a 1:1 duel).**

$$\vec{x}_i(t+1) = \begin{cases} \vec{u}_i & \text{if } f(\vec{u}_i) \le f(\vec{x}_i(t)), \\ \vec{x}_i(t) & \text{otherwise.} \end{cases}$$

Selection is therefore elitist at the level of the individual — the population never gets worse.

Algorithm 6 DE/rand/1/bin

Input: population size N, scale factor F, crossover threshold C

initialise the population $\{\vec{x}_i\}_{i=1}^{N}$ at random within the bounds of the problem
while the termination condition is not met **do**
    for $i \leftarrow 1$ **to** N **do**
        select distinct random indices $a, b, c \neq i$
        $\vec{v} \leftarrow \vec{x}_a + F(\vec{x}_b - \vec{x}_c)$
        $I_{\text{rand}} \leftarrow$ a random index from $\{1, \ldots, D\}$
        for $j \leftarrow 1$ **to** D **do**
            $u_j \leftarrow v_j$ if rand() $\le C$ or $j = I_{\text{rand}}$, otherwise $u_j \leftarrow x_{j,i}$
        end for
        if $f(\vec{u}) \le f(\vec{x}_i)$ **then** $\vec{x}'_i \leftarrow \vec{u}$
        else $\vec{x}'_i \leftarrow \vec{x}_i$
        end if
    end for
    $\{\vec{x}_i\} \leftarrow \{\vec{x}'_i\}$
end while

3.2.3 A numerical example of one step

Let $D = 3$ and let us minimise $f(\vec{x}) = \sum_j x_j^2$ with the parameters $F = 0.8$, $C = 0.9$. The current individual is $\vec{x}_i = (4;\ -3;\ 2)$, $f(\vec{x}_i) = 16 + 9 + 4 = 29$. The randomly selected individuals are:

$$\vec{x}_a = (2;\ -1;\ 0.5), \quad \vec{x}_b = (1;\ 0;\ -1), \quad \vec{x}_c = (-1;\ 2;\ 1).$$

Mutation:

$$\vec{x}_b - \vec{x}_c = (2;\ -2;\ -2), \qquad \vec{v} = (2;\ -1;\ 0.5) + 0.8\,(2;\ -2;\ -2) = (3.6;\ -2.6;\ -1.1).$$

Crossover: let $I_{\text{rand}} = 1$ and let the random numbers be rand $= (0.40;\ 0.95;\ 0.20)$. Then

- $j = 1$: $0.40 \le 0.9$ (and moreover $j = I_{\text{rand}}$) $\Rightarrow u_1 = v_1 = 3.6$;
- $j = 2$: $0.95 > 0.9$ and $j \neq I_{\text{rand}} \Rightarrow u_2 = x_{2,i} = -3$;
- $j = 3$: $0.20 \le 0.9 \Rightarrow u_3 = v_3 = -1.1$.

The trial vector is $\vec{u} = (3.6;\ -3;\ -1.1)$.

Selection: $f(\vec{u}) = 12.96 + 9 + 1.21 = 23.17 < 29 = f(\vec{x}_i)$, so $\vec{u}$ replaces $\vec{x}_i$ in the new population.

3.2.4 Mutation variants

The notation `DE/`x`/`y`/`z: x = how the base vector is chosen, y = the number of difference vectors, z = the type of crossover (`bin` = binomial uniform, `exp` = exponential, a contiguous segment).

variant	**donor** $\vec{v}$
DE/rand/1	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c)$
DE/rand/2	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c + \vec{x}_d - \vec{x}_e)$
DE/best/1	$\vec{x}_{\text{best}} + F(\vec{x}_b - \vec{x}_c)$
DE/best/2	$\vec{x}_{\text{best}} + F(\vec{x}_b - \vec{x}_c + \vec{x}_d - \vec{x}_e)$
DE/current-to-best/1	$\vec{x}_i + F(\vec{x}_{\text{best}} - \vec{x}_i) + F(\vec{x}_b - \vec{x}_c)$

The variants with `best` converge faster, but they get stuck in a local optimum more easily; `rand` is more robust. The modern variants (jDE, SaDE, JADE, SHADE, L-SHADE) adapt F and C during the run and are among the top performers in the CEC competitions.

3.2.5 Comparison of DE with ES and GA

	GA (real-valued)	**ES**	**DE**
source of the mutation step	fixed σ	evolved σ	differences of individuals
parent selection	according to fitness	random	every individual once
env. selection	generational / elitism	the μ best	duel parent vs. offspring
main operator	crossover	mutation	differential mutation
adaptation of the scale	none	explicit (in the genome)	implicit (from the population)

3.3 Coevolution

3.3.1 Contextual fitness

Coevolution

An evolutionary algorithm in which the fitness of an individual is **not absolute** but depends on the other individuals — on its own population or on another population evolving alongside it. Fitness is therefore *contextual*, and the fitness landscape is **dynamic**: it changes as the opponents or the partners change.

Motivation: for many problems we cannot write down an absolute objective function ("how good is a strategy for draughts?"), but we can *compare* two individuals (by playing a game). In addition, coevolution promises *evolutionary arms races* — the opponents push each other towards ever better solutions. Types of coevolution:

	competitive	**cooperative**
relationship	the fitness of one grows at the expense of the other	the individuals complement each other into a common solution
example	predator–prey, host–parasite, player–opponent, sorting networks vs. sequences of numbers	decomposition of a problem into subproblems, modules and blueprints (CoDeepNEAT), main programs and ADFs
fitness	the outcome of the duel (the tournament)	the quality of the composite solution the individual took part in
risk	cycling, disengagement, overspecialisation	credit assignment, lazy partners

3.3.2 Competitive coevolution

The classical example: Hillis's sorting networks (1990). A population of sorting networks coevolves against a population of test sequences. The fitness of a network = the proportion of correctly sorted tests; the fitness of a test = how many networks failed on it. The parasitic tests specialise in the weaknesses of the networks, and the networks have to improve. In this way Hillis found a network for 16 elements with **61** comparators. *Beware of a frequent mistake:* the best known *human* design (Green, 1969) has 60 comparators, so coevolution did not beat it. What matters is the comparison within the experiment — the same genetic algorithm *without* coevolving parasites

achieved only 65 comparators. Coevolution therefore demonstrably improved the optimisation ability of the EA and came within a single comparator of the human result.

Predator–prey in evolutionary robotics: hunter robots and fugitive robots perfect each other. **Host–parasite**: models in artificial life (Tierra), software testing.

Tournament evaluation. Fitness is obtained from duels: round robin (expensive), a random sample of opponents (cheap, noisy), a *hall of fame* (a duel against an archive of the historically best individuals — it prevents forgetting), or a *tournament with shared fitness* (*competitive fitness sharing*: defeating an opponent whom few can defeat brings a higher reward).

3.3.3 Pathologies of coevolution

- **Cycling** (intransitivity): the populations go round in a circle as in rock–paper–scissors; the apparent progress is not real. The dynamics of coevolution can even be deterministically chaotic.
- **Disengagement**: one population completely outgrows the other, all duels end the same way, fitness loses its discriminating power and selection stops working (the gradient disappears).
- **The Red Queen effect**: everyone improves, but the relative fitness stays constant — from the measured values we cannot tell whether the progress is real. The remedy: measuring against a fixed external set of opponents (a *master tournament*).
- **Mediocre stable states**: the populations settle into a mutual equilibrium at a low level.
- **Overspecialisation** (*focusing*): an individual optimises itself against the current opponent and loses generality.
- **Forgetting**: the ability to defeat old opponents is lost. The remedy: archives (hall of fame, Pareto-coevolutionary archives, *Nash memory*).

3.3.4 Cooperative coevolution

CCGA (cooperative coevolutionary GA, Potter & De Jong)

The problem is split into k components (e.g. coordinates, modules, rules). A **separate population** is maintained for each component. The fitness of an individual from population i is determined by *combining* it with representatives of the other populations (typically with their currently best individuals, possibly also with random ones) and evaluating the resulting composite solution.

Advantages: the decomposition of a large space into smaller ones, natural parallelisation, suitability for modular problems. Problems: *credit assignment* — how to divide the quality of the composite solution among the components; *interdependence* (strongly interacting components cannot be separated well); *relative overspecialisation* (a population adapts to particular partners).

Examples in practice: the coevolution of main programs and ADFs in GP, CoDeepNEAT (a population of modules and a population of "blueprints", section 6.2), meta-Lamarckian memetic algorithms (a population of memes and a population of solutions, chapter 5).

3.3.5 The evolution of cooperation and the prisoner's dilemma

Coevolution is closely related to a basic question of evolutionary biology: *how can altruism arise* when by definition it lowers the fitness of the individual? Darwinism is competitive in its essence ("survival of the fittest"), and yet there is a great deal of cooperation both in nature and in society. The theories:

- **Group selection** — evolution acts on groups as well (Darwin); the problem: how to explain cheaters inside the group.

- **Kin selection** — the preservation of almost identical genes in relatives; the problem: altruism towards strangers and other species.
- **The selfish gene** (Dawkins) — the unit of evolution is the gene, not the individual. (Wilson: "an organism is only DNA's way of making more DNA".)
- **Reciprocal altruism** (Trivers, 1971) — mutual benefit, the *shadow of the future*; the formal model = the iterated prisoner's dilemma.

The prisoner's dilemma

A two-player game with the actions C (cooperate) and D (defect) and the payoffs

	C	D
C	R/R	S/T
D	T/S	P/P

$T > R > P > S$ (and for the iterated version $2R > T + S$).

Typically $R = -1$, $T = 0$, $P = -2$, $S = -3$. Because $T > R$ and $P > S$, defection is the **dominant strategy** of both players; DD is a **Nash equilibrium**, but as the only outcome it **is not Pareto-optimal** — CC maximises the joint gain.

Rational players thus arrive at an outcome that is worse for both of them than the cooperation they could have achieved. The same mechanism in its n-player form is called the *tragedy of the commons*: it individually pays everyone to draw a little more from a shared resource, and the resource therefore perishes. The question "what is actually rational (and do people behave that way?)" is at the heart of the whole discussion around the prisoner's dilemma.

Dominant strategy, Nash equilibrium, Pareto optimality

A strategy s is *dominant* for player i if it gives the same or a better outcome than any other strategy of i against *all* strategies of the opponent. A pair (s_i, s_j) is a *Nash equilibrium* if the strategies are mutual best responses (no player has a reason to change strategy unilaterally). A solution is *Pareto-optimal* if the outcome of one player cannot be improved without worsening the outcome of another. Nash's theorem: every game with finitely many strategies has an equilibrium in *mixed* strategies (a random choice among the pure ones) — e.g. rock–paper–scissors.

The iterated version and Axelrod's tournaments. If the game is played repeatedly and the players remember the history, cooperation may pay off (the *shadow of the future*). If the exact number of rounds N is known in advance, backward induction shows that it is optimal to keep defecting — in practice, however, the players do not know the end. Axelrod (1980) organised tournaments of strategies:

- 1st tournament: 14 strategies + RANDOM, 200 games, round robin (including against itself), 5 repetitions. **TFT** (*tit for tat*) won: start by cooperating, then repeat the opponent's last move.
- 2nd tournament: 62 strategies, everyone knew the results of the first — TFT won again.
- 3rd, the "ecological" tournament: a simulation as in a GA — the number of individuals of a strategy in the next generation is proportional to the number of wins, 1000 generations. TFT prevailed once more.

Four properties of successful strategies: *niceness* (do not defect first), *provocability* (retaliation — punish a defection immediately), *forgiveness* (but calm down quickly), *non-enviousness* (do not strive to beat a *particular* opponent; TFT never scored more points than its opponent, and yet it won the tournament). As a fifth property Axelrod lists *clarity* (be simple, so that others are able to establish cooperation with you at all). No strategy wins against everybody; one has to be successful against very diverse opponents (ALL-D, TFTT, RANDOM, TRIGGER) and to play well against oneself.

Lessons and later developments. In environments where the payoffs favour cooperation and where the probability of repetition is high, cooperation usually evolves — and neither rationality nor consciousness is needed for that, higher fitness suffices. In a noisy environment (a move executed incorrectly) the more successful strategy is **Pavlov** (*win-stay, lose-shift*): if I received R or P, repeat the move (C); if I received T or S, change it (D). The tournament repeated after 20 years was won by a *team* of strategies from the University of Southampton: the first ten moves or so served to recognise a teammate, after which all the members of the team selflessly helped one "parent" strategy to obtain a high score.

3.3.6 Diversity, species and niches

A small selection pressure is enough for the diversity of a population to disappear. (A similar line of thought is developed by Flegr's *frozen evolution*: a genetically variable species behaves flexibly towards selection only for a short time, after which it "freezes" and stops responding to changes of the environment.) Mechanisms for maintaining diversity (useful for coevolution as well as for dynamic fitness and multi-objective optimisation):

- **Fitness sharing:** fitness is divided by the number of similar individuals,

$$f'(i) = f(i) \Big/ \sum_j sh\big(d(i,j)\big), \qquad sh(d) = 1 - (d/\sigma_{\text{share}})^\alpha$$

  for $d < \sigma_{\text{share}}$ and 0 otherwise. It creates pressure towards occupying different niches.
- **Crowding:** a new individual replaces the *most similar* individual (deterministic crowding, RTS).
- **Non-random mating:** crossover only between similar individuals (*speciation*), or according to a topology — the individuals live on a 2D/3D grid and mate only with their neighbours.
- **The island model:** several subpopulations with occasional migration (in more detail below).
- **Explicit species** by means of tag bits, or according to the structural similarity of the genomes (NEAT, section 6.2); mating is then allowed only within a species.

The problem is always the definition of similarity (Hamming distance, distance in the phenotype space, the number of matching innovation numbers) and the choice of the niche threshold.

Parallel models of EAs. The structure of the population is closely related to parallelisation; three models are distinguished:

- **Master–slave** (global parallelisation): a single population, and only the *fitness evaluation* is distributed among the nodes. The algorithm is mathematically identical with the sequential one; it pays off when fitness is expensive.
- **The island / coarse-grained model**: several independent subpopulations between which a few individuals occasionally *migrate*. Parameters: the topology of the islands, the *migration interval* and the *migration rate*, the choice of the migrants and of those they replace. Rare migration maintains diversity (the islands converge to different places); if it is too frequent, the model degenerates into a single population.
- **The cellular / fine-grained (diffusion) model**: the individuals sit on a 2D/3D grid and mate only with their neighbours. A good solution spreads through the population like a wave, which strongly slows down premature convergence; this is an implicit form of non-random mating.

The island and the cellular model are therefore not merely an implementation trick — they change the very dynamics of the algorithm and belong among the standard tools for maintaining diversity.

3.3.7 Dynamic fitness landscapes

Fitness landscape

Introduced by Sewall Wright (1932). A graphical representation of the dependence of fitness on the genotype (or on the allele frequencies or on a phenotypic trait). A theoretical tool for reasoning about EAs; in higher dimensions, however, it is not intuitive.

Fitness often changes in time: a robot in a room where somebody turns the light on; it starts raining; a stock market crash; tournaments of agents (coevolution). Classical GAs perform poorly, because they are "overfitted" to static problems — they converge quickly and lose the diversity they would need in order to react to the change. De Jong's comparison: a $(1+10)$-ES reacts slowly to a sudden change, because self-adaptation has shrunk its step; a generational GA with roulette wheel selection and without elitism keeps more diversity and recovers faster.

Modifications of EAs for dynamic landscapes:

- **Diploid representation** (Goldberg, Smith) — two sets of chromosomes with dominance act as a long-term memory when the fitness oscillates.
- **Hypermutation** (Cobb, Grefenstette) — if the average fitness of the population drops (the environment has probably changed), the mutation probability is raised considerably.
- **Random immigrants** — a batch of new random individuals is inserted into the population.
- **Maintaining diversity** — a lower selection pressure, crowding, niching, subpopulations, the comma ES (μ, λ) (the parents do not survive, so the population detaches itself from an obsolete optimum more easily).

Types of change: small and smooth (wear of the hardware), morphological (hills appear and disappear), cyclic (the seasons), catastrophic and discontinuous. If the fitness changes too fast, no solution is permanently good (cf. No Free Lunch).

3.4 Open-ended evolution

[♦ beyond the slides: the whole section except interactive evolution] The topic names open-ended evolution explicitly, but neither the EVA I nor the EVA II slides contain anything on it, and *Evolutionary Robotics* mentions it in a single sentence ("the evolution of robots is a continuous process with an open end"). It is therefore treated briefly, at a level that will do at the examination. The exception is *interactive evolution* at the end of the section — that one is in the Evolutionary Robotics slides (subjective fitness, Biomorphs).

Open-ended evolution

An evolutionary process that **has no fixed goal** and that keeps producing new complexity and new "kinds" of behaviour without converging. The model is the terrestrial biosphere: it has no objective function, and yet it permanently generates novelty.

The hallmarks of open-endedness. In order to be able to decide about a system at all, *hallmarks* are formulated: **the sustained production of novelty** (new behaviour appears even after an arbitrarily long run); **the unbounded growth of complexity** of the individuals and of their relationships; **the emergence of new classes of entities** and levels of organisation (in biology the *major transitions*: replicator → cell → multicellularity → society); **the evolution of evolvability** — the very way in which the system generates novelty changes as well.

The systems and why they get stuck. *Tierra* (Ray) and *Avida* — self-copying machine-code programs in which parasites and hyperparasites emerged; *Polyworld* — agents with neural networks in a 2D world; *ECHO* (Holland) — agents with "tags" and resources. Practically every one of them reaches, after some time, a state in which nothing qualitatively new appears any more: the

environment is finite, the space of possible "inventions" is exhausted and the dynamics freezes. The hypotheses as to why: the lack of a sufficiently rich and self-extending environment, the lack of a genuinely open representation, and the lack of a mechanism that would continually raise the demands placed on the individuals. Achieving genuinely open-ended evolution remains an open research problem.

Generative (indirect) representations. Related to open-ended evolution is the question of how to encode unboundedly growing complexity at all. The answer are encodings in which the genotype is not a description of the result but **a recipe for growing the result**: *L-systems* (rewriting grammars generating branching structures), *cellular automata*, *cellular encoding* (Gruau) and *CPPN/HyperNEAT* for neural networks (section 6.2). Their common contribution: a small genome, a large phenotype, and a natural expression of symmetries and of repeated motifs.

Novelty search and sparseness

A radical idea (Lehman & Stanley, 2011: *Abandoning Objectives*): in deceptive problems the objective function is misleading — it leads into local optima. Therefore **let us throw the goal away and reward novelty alone**.
A *behaviour space* is defined (e.g. the final position of a robot in a maze) and the novelty of an individual x is measured by the sparseness of its neighbourhood,

$$\rho(x) \;=\; \frac{1}{k}\sum_{i=1}^{k} \mathrm{dist}\big(x, \mu_i\big),$$

where the μ_i are the k nearest neighbours of x in the current population **and in a permanent archive**. Fitness $= \rho$; the original objective function *is not used at all*. On problems of the "robot in a maze" type it tends to be more successful than goal-directed optimisation. It is followed up by **quality-diversity** (MAP-Elites): a grid of cells indexed by behaviour descriptors, with the best solution found in each cell.

Interactive evolution. The fitness function is replaced by a **human being**: the user is presented with a population of candidates and picks the ones they like. It is used where the quality criterion cannot be written down (aesthetics, design, music). Examples: Dawkins's *Biomorphs*, **Picbreeder** and **EndlessForms** (the evolution of pictures and of 3D shapes by means of CPPNs). The limitation is slowness and user fatigue — small populations are necessary (9–20 individuals per screen) and few generations. It was precisely the experience from Picbreeder (the most interesting objects were never produced by deliberate search) that directly motivated novelty search.

3.5 Exam summary

- ES: real-valued vectors, mutation the main operator, random parent selection, deterministic environmental selection; an individual $= [\vec{x}, \vec{\sigma}]$.
- (μ, λ) vs. $(\mu + \lambda)$: be able to list the differences (elitism, convergence, local optima, self-adaptation).
- $(1 + 1)$-ES and the 1/5 rule: $p > 1/5 \Rightarrow$ increase σ, $p < 1/5 \Rightarrow$ decrease it; this is adaptive, not self-adaptive control.
- Self-adaptation: $\sigma' = \sigma \exp(\tau' N(0,1) + \tau N_i(0,1))$, then $x' = x + \sigma' N(0,1)$; **first σ, then x**. The number of strategy parameters: 1 (isotropic), n (uncorrelated), $n(n+1)/2$ (correlated, with rotation angles).
- Recombination: local ($\rho = 2$) / global ($\rho = \mu$), discrete (dominant) / intermediate (arithmetic).
- CMA-ES: $\theta = (\vec{m}, \sigma, \mathbf{C}, \vec{p}_\sigma, \vec{p}_c)$, sampling from $\mathcal{N}(\vec{m}, \sigma^2\mathbf{C})$, evolution paths, rank-1 and rank-μ updates, step control according to the length of $\vec{p}_\sigma$; invariant with respect to monotone transformations of fitness and to linear transformations of the space.

- DE/rand/1/bin: the donor $\vec{v} = \vec{x}_a + F(\vec{x}_b - \vec{x}_c)$; binomial crossover with the threshold C and the safeguard $j = I_{\text{rand}}$; a duel of the offspring with its own parent (replacement only upon improvement).
- Parameters: $F \in [0, 2]$ (typically 0.5–0.9), $C \in [0, 1]$, $N \approx 10D$.
- The geometric essence: the scale and the orientation of the steps are taken over from the current spread of the population ⇒ automatic adaptation without strategy parameters.
- Variants: rand/1, rand/2, best/1, best/2, current-to-best; `bin` vs. `exp` crossover.
- Be able to demonstrate one step with concrete numbers (mutation → crossover → selection).
- Coevolution = contextual fitness + a dynamic landscape. *Competitive*: predator–prey, host–parasite, Hillis's sorting networks. *Cooperative*: decomposition into components, CCGA, modules + blueprints.
- Pathologies: cycling (intransitivity), disengagement, the Red Queen effect, mediocre stable states, overspecialisation, forgetting. The remedies: archives (hall of fame), shared fitness, measurement against a fixed set.
- The prisoner's dilemma: $T > R > P > S$, $2R > T + S$; D is dominant, DD is a Nash equilibrium and the only outcome that is not Pareto-optimal; the iterated version ⇒ TFT; the four properties of a successful strategy (niceness, provocability, forgiveness, non-enviousness) + clarity as the fifth one; Pavlov in a noisy environment.
- Diversity: fitness sharing, crowding, speciation, the island model, non-random mating.
- Dynamic fitness: diploidy, hypermutation, random immigrants, maintaining diversity, the (μ, λ)-ES.
- Open-ended evolution: without a fixed goal; the hallmarks = sustained novelty, unbounded growth of complexity, new classes of entities, the evolution of evolvability. The systems Tierra, Avida, Polyworld, ECHO — all of them get stuck sooner or later. Generative (indirect) representations: L-systems, cellular automata, cellular encoding, CPPNs.
- Novelty search measures the sparseness $\rho(x)$ in the behaviour space with respect to the population *and* a permanent archive, and does not use the objective function at all; it is followed up by quality-diversity and MAP-Elites (a grid of behaviour descriptors).
- Interactive evolution: fitness = a human being (Biomorphs, Picbreeder, EndlessForms); the limitations are the small population and the user's fatigue.

4 Swarm optimisation algorithms

Swarm intelligence

An approach in which globally intelligent behaviour **emerges** from simple local rules of many individuals without any central control. The features: decentralisation, simpler individuals, indirect communication through the environment (*stigmergy*), robustness against the failure of an individual. Unlike in EAs, there is usually no crossover, no mutation and no "death" of the individuals here — the individuals move and remember.

4.1 Particle swarm optimisation (PSO)

Particle swarm optimisation (Eberhart and Kennedy, 1995) is inspired by flocks of birds and schools of fish. An individual is a *particle* — a point in $\mathbb{R}^n$ with a velocity. Neither crossover nor mutation is used; instead the algorithm has a **memory**:

- $\vec{p}_i$ (*pBest*) — the best position particle i has ever visited (the cognitive, individual memory),
- $\vec{g}$ (*gBest*) — the best position found by the whole swarm (the social, collective memory).

The PSO equations of motion

$$\vec{v}_i \leftarrow \underbrace{w\,\vec{v}_i}_{\text{inertia}} + \underbrace{c_1 r_1(\vec{p}_i - \vec{x}_i)}_{\text{cognitive component}} + \underbrace{c_2 r_2(\vec{g} - \vec{x}_i)}_{\text{social component}}, \qquad \vec{x}_i \leftarrow \vec{x}_i + \vec{v}_i,$$

where $r_1, r_2 \sim U(0,1)$ (drawn separately for each component), c_1, c_2 are the learning coefficients (in the original version $c_1 = c_2 = 2$) and w is the *inertia weight*. In the original 1995 version $w = 1$ (it is missing); Shi and Eberhart added it and recommend a linear decrease, e.g. from 0.9 to 0.4: a large w supports exploration, a small one exploitation.

Algorithm 7 PSO

initialise the positions $\vec{x}_i$ and the velocities $\vec{v}_i$ at random; $\vec{p}_i \leftarrow \vec{x}_i$
repeat
  for each particle i **do**
    evaluate $f(\vec{x}_i)$
    if $f(\vec{x}_i)$ is better than $f(\vec{p}_i)$ **then** $\vec{p}_i \leftarrow \vec{x}_i$
    end if
  end for
  $\vec{g} \leftarrow$ the best of $\{\vec{p}_i\}$
  for each particle i **do**
    update the velocity and the position according to the equations of motion
  end for
until the maximum number of iterations or the required precision is reached

A numerical example. A particle in $\mathbb{R}^2$: $\vec{x} = (1; 2)$, $\vec{v} = (0.5; -0.5)$, $\vec{p} = (0; 3)$, $\vec{g} = (-1; 1)$, $w = 0.7$, $c_1 = c_2 = 1.5$, $r_1 = 0.4$, $r_2 = 0.8$.

$$\begin{aligned}\vec{v}' &= 0.7\,(0.5;\,-0.5) + 1.5 \cdot 0.4\,\big((0;3) - (1;2)\big) + 1.5 \cdot 0.8\,\big((-1;1) - (1;2)\big)\\ &= (0.35;\,-0.35) + 0.6\,(-1;\,1) + 1.2\,(-2;\,-1) = (-2.65;\,-0.95),\\ \vec{x}' &= (1;\,2) + (-2.65;\,-0.95) = (-1.65;\,1.05).\end{aligned}$$

The particle has thus overshot $\vec{g}$ — typical PSO behaviour, and the source of its exploration. That is why the velocity is bounded ($|v_j| \le v_{\max}$) or a *constriction factor* (Clerc and Kennedy) is used: $\vec{v} \leftarrow \chi[\vec{v} + c_1 r_1(\vec{p} - \vec{x}) + c_2 r_2(\vec{g} - \vec{x})]$, where $\chi = 2/|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}|$ for $\varphi = c_1 + c_2 > 4$; for $c_1 = c_2 = 2.05$ this gives $\chi \approx 0.729$. This choice guarantees the convergence of the swarm.

Neighbourhood topologies. Instead of a global $\vec{g}$ (the *gbest* topology, i.e. the complete graph) one can use *lbest* — the best individual in a local neighbourhood:

- **A ring** (each particle has k neighbours): information spreads slowly ⇒ slower convergence, but better exploration and a smaller risk of getting stuck.
- **A von Neumann grid**, **a star**, **random graphs**.
- Empirically: gbest is faster on simple problems, lbest is more robust on multimodal ones.

Variants and the relation to EAs. Discrete/binary PSO (the velocity is interpreted as the probability of flipping a bit via a sigmoid), PSO with several swarms, hybridisation with local search, *grammatical swarm*.

	genetic algorithm	PSO
in common	random initialisation, fitness, stochastic updates	the same
individuals	are born and die	exist permanently, they move
memory	only in the population	explicit ($\vec{p}_i$, $\vec{g}$)
operators	crossover, mutation	none, only movement
information flow	two-way between the parents	one-way: from the better ones to the rest

4.2 Ant colony optimisation (ACO)

[♦ beyond the slides: of the swarm algorithms the EVA slides cover only PSO; the topic, however, speaks of swarm algorithms in the plural and ACO is their canonical second representative]

Ant colony optimisation (M. Dorigo, 1991) imitates the way ants find a path: an ant lays down a *pheromone trail*, along a shorter path it returns sooner, the pheromone accumulates there faster, and that attracts further ants — positive feedback. Moreover the pheromone **evaporates**, so the system forgets obsolete information and does not get stuck.

Stigmergy

Indirect communication by means of changes in the environment. The ants do not communicate with each other directly; the only channel is the pheromone on the edges. A general principle of reactive multiagent architectures.

An illustration — Steels's Mars explorer. A swarm of robots is to collect rock samples; the base emits a navigation signal (it suffices to follow the gradient). A robot has "crumbs" (radioactive markers) for indirect communication. The rules, with priorities: R1: if you see an obstacle, change direction; R2: if you are carrying a sample and you are at the base, put it down; R3: if you are carrying a sample, go up the gradient; R4: if you see a sample, pick it up; R5: otherwise move at random. The second iteration adds stigmergy: R7: if you are carrying a sample and you are not at the base, drop 2 crumbs and go up the gradient; R8: if you sense a crumb, pick up 1 crumb and go down the gradient. What arises is an effective collective collection from rich deposits, without the robots having either a map or communication.

The general ACO scheme. The problem is transformed into finding a path in a weighted graph; the ants are agents constructing solutions along the edges.

1. Every ant stochastically constructs a solution (a sequence of edges).
2. Optionally *daemon actions* — a centralised step, typically a local search improving the paths found.
3. The qualities of the solutions are compared.
4. The pheromones on the edges are updated (evaporation + deposition).

Ant System (AS) — the transition rule

Ant k in city i chooses the next city j from the set of the not yet visited ones $\mathcal{N}_i^k$ with probability

$$p_{ij}^k = \frac{\left[\tau_{ij}\right]^\alpha \left[\eta_{ij}\right]^\beta}{\sum_{l \in \mathcal{N}_i^k} \left[\tau_{il}\right]^\alpha \left[\eta_{il}\right]^\beta}, \qquad \eta_{ij} = \frac{1}{d_{ij}},$$

where τ_{ij} is the amount of pheromone on the edge, η_{ij} is the heuristic "visibility" (the reciprocal distance) and α, β are the weights of the two influences (typically $\alpha = 1$, $\beta \in [2, 5]$).

Ant System — the pheromone update

$$\tau_{ij} \leftarrow (1-\rho)\,\tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^{k}, \qquad \Delta\tau_{ij}^{k} = \begin{cases} Q/L_k & \text{if ant } k \text{ used the edge } (i,j), \\ 0 & \text{otherwise,} \end{cases}$$

where $\rho \in (0,1)$ is the evaporation rate and L_k the length of the tour of ant k. A shorter tour $\Rightarrow$ more pheromone.

A numerical example (the transition rule). An ant stands in city i and can go to the cities A, B, C with distances $d = (5; 10; 2)$ and pheromones $\tau = (2; 4; 1)$; $\alpha = 1$, $\beta = 2$. The visibilities: $\eta = (0.2; 0.1; 0.5)$. The weights:

$$w_A = 2 \cdot 0.2^2 = 0.08, \quad w_B = 4 \cdot 0.1^2 = 0.04, \quad w_C = 1 \cdot 0.5^2 = 0.25, \quad \sum = 0.37.$$

The probabilities: $p_A = 0.216$, $p_B = 0.108$, $p_C = 0.676$. Even though the edge to B has the most pheromone, the short distance to C (raised to the power $\beta = 2$) is what decides.

A numerical example (the update). Let $\tau_{ij} = 2$, $\rho = 0.5$, $Q = 1$, and let two ants with tour lengths $L_1 = 20$, $L_2 = 25$ have used the edge:

$$\tau_{ij} \leftarrow 0.5 \cdot 2 + \left(\tfrac{1}{20} + \tfrac{1}{25}\right) = 1 + 0.09 = 1.09.$$

ACS (Ant Colony System). An improvement competitive with specialised TSP solvers; inspired by Q-learning:

- **Global update by the best solution only** (elitism): pheromone is added only by the globally (or iteration-) best ant.
- **Local update**: whenever an ant uses an edge, the pheromone on it is slightly decreased ($\tau \leftarrow (1-\xi)\tau + \xi\tau_0$) — the edge thereby becomes less attractive to the others and the variety of the tours grows.
- **A pseudo-random selection rule**: with probability q_0 the best edge according to $\tau_{il}\eta_{il}^{\beta}$ is chosen deterministically (exploitation), otherwise the standard AS rule is used (exploration).

Other variants. *Elitist AS* (the best tour contributes extra), *Rank-based AS* (the N best contribute, with a weight according to the rank), **MAX–MIN AS** (the pheromone is kept in the interval $[\tau_{\min}, \tau_{\max}]$, only the best one updates, initialisation to $\tau_{\max}$ — this prevents premature stagnation), ACO for continuous optimisation (the space is discretised into subintervals and the pheromone guides the search into better subspaces).

Properties and applications. Decentralisation, emergence, indirect communication; close to reactive agent architectures (Brooks's subsumption architecture). A great advantage is the ability to **optimise in a dynamic environment** — when the graph changes, the pheromones adapt. Applications: the TSP, vehicle routing, routing in networks, DNA sequencing, protein folding, edge detection in images, scheduling.

4.3 Other swarm algorithms

[♦ beyond the slides]

- **The artificial bee colony** (ABC; Karaboga 2005). The population of food sources = the solutions. Three roles: *employed bees* (they search the neighbourhood of their source), *onlookers* (they choose a source by the roulette wheel according to its quality and search its neighbourhood), *scouts* (a source that has not improved for *limit* attempts is abandoned and replaced by a random one — a built-in restart).

- **The firefly algorithm** (Yang, 2008). The individuals = fireflies; the brightness is proportional to fitness and decreases with distance as $I(r) = I_0 e^{-\gamma r^2}$. A less bright firefly moves towards a brighter one: $\vec{x}_i \leftarrow \vec{x}_i + \beta_0 e^{-\gamma r_{ij}^2}(\vec{x}_j - \vec{x}_i) + \alpha\vec{\varepsilon}$. Thanks to the limited range of the attraction the swarm naturally splits into subgroups ⇒ suitable for multimodal problems.
- **The bat algorithm**, **cuckoo search**, **the grey wolf optimizer** and dozens of other "metaphorical" algorithms. The criticism: many of them are merely renamed variants of PSO or ES and bring terminological confusion rather than new principles.

4.4 Exam summary

- PSO: the equations $\vec{v} \leftarrow w\vec{v} + c_1 r_1(\vec{p} - \vec{x}) + c_2 r_2(\vec{g} - \vec{x})$, $\vec{x} \leftarrow \vec{x} + \vec{v}$; the inertia w (decreasing 0.9 → 0.4), the cognitive and the social component, $c_1 = c_2 = 2$; the bound $v_{\max}$ or the constriction $\chi \approx 0.729$; the gbest vs. lbest topology (ring, grid).
- The difference between PSO and a GA: no operators, the particles have a memory ($\vec{p}_i$, $\vec{g}$), the information flows one way from the better ones.
- ACO: the transition probability $p_{ij} \propto \tau_{ij}^{\alpha}\eta_{ij}^{\beta}$ with $\eta = 1/d$; the update $\tau \leftarrow (1-\rho)\tau + \sum_k Q/L_k$ (evaporation + deposition proportional to the quality).
- AS (the original one, everybody updates) vs. ACS (global update by the best one only, local update on traversal, the pseudo-random rule q_0) vs. MAX–MIN (bounds on the pheromone). Daemon actions = local search.
- ABC (employed bees / onlookers / scouts with a limit = restart), firefly (the attraction decreases exponentially with the distance).
- Keywords: stigmergy, emergence, decentralisation, positive feedback + evaporation, suitability for dynamic environments.

5 Local search and memetic algorithms

5.1 Local search and hill climbing

Neighbourhood and local search

For a solution space X we define a *neighbourhood* $N(x) \subseteq X$ — the set of solutions reachable from x by one elementary change (flipping a bit, swapping two cities, a 2-opt step). *Local search* iteratively moves to a neighbour according to a given rule. A *local optimum* with respect to N is an x such that no neighbour is better — so it depends on the definition of the neighbourhood!

Hill climbing

1. Steepest ascent: traverse the *whole* neighbourhood and move to the best neighbour if it is better than x; otherwise stop.
2. First improvement: traverse the neighbourhood and move to the first better neighbour found (cheaper, often just as good).
3. Stochastic hill climbing: pick a random neighbour; accept it if it is better (or with a probability proportional to the improvement).
4. Random-restart hill climbing: once stuck, start from a new random position; remember the best solution found. As the number of restarts grows, the probability of finding the global optimum converges to 1.

Weaknesses: getting stuck in a local optimum, on *plateaux* (where all the neighbours are the same — *sideways moves* help) and on ridges. Advantages: simplicity, small memory, speed — which is why it is ideal as an improvement procedure inside an EA.

5.2 Simulated annealing

Simulated annealing (SA)

Kirkpatrick, Gelatt, Vecchi (1983); a physical analogy with the slow cooling of a metal, during which the atoms take up an arrangement of minimal energy. The algorithm is a hill climber that **also admits worsening steps** with a probability that decreases with the size of the worsening and with the temperature.

The Metropolis criterion

For minimisation, a candidate $y \in N(x)$ and $\Delta = f(y) - f(x)$:

$$P(\text{accept } y) = \begin{cases} 1, & \Delta \le 0 \quad \text{(an improvement)}, \\ e^{-\Delta/T}, & \Delta > 0 \quad \text{(a worsening)}, \end{cases}$$

where $T > 0$ is the temperature.

A numerical example. A worsening of $\Delta = 2$. For $T = 1$ we get $P = e^{-2} = 0.135$; for $T = 0.5$, $P = e^{-4} = 0.018$; for $T = 5$, $P = e^{-0.4} = 0.67$. At the beginning (hot) the algorithm therefore behaves almost like a random walk, at the end (cold) like a pure hill climber.

Algorithm 8 Simulated annealing

$x \leftarrow$ an initial solution; $T \leftarrow T_0$; $x^* \leftarrow x$
while $T > T_{\min}$ and the termination condition is not met **do**
  for k times (the length of the Markov chain at the given temperature) **do**
    pick $y \in N(x)$ at random; $\Delta \leftarrow f(y) - f(x)$
    if $\Delta \le 0$ or rand() $< e^{-\Delta/T}$ **then** $x \leftarrow y$
    end if
    if $f(x) < f(x^*)$ **then** $x^* \leftarrow x$
    end if
  end for
  $T \leftarrow \text{cool}(T)$
end while
Output: x^*

The cooling schedule.

- *Geometric:* $T_{k+1} = \alpha T_k$, $\alpha \in [0.8, 0.99]$ — the most common one in practice.
- *Linear:* $T_k = T_0 - \beta k$.
- *Logarithmic:* $T_k = c/\log(k+1)$ — the only one for which convergence has been proved.
- Adaptive (reheating: the temperature is raised upon stagnation).

Theorem 5.1 (Convergence of SA). *With a logarithmic schedule $T_k \ge c/\log(k+1)$ with a sufficiently large constant c (related to the height of the deepest "barrier" in the landscape), simulated annealing converges to the set of global optima with probability 1.*

In practice this schedule is unusably slow; SA is therefore theoretically satisfactory, but faster heuristic schedules are used in practice. Formally SA is an inhomogeneous Markov chain; for a fixed T it converges to the Boltzmann distribution $\pi(x) \propto e^{-f(x)/T}$, which for $T \to 0$ concentrates on the global optima. The connection with EAs: the *Boltzmann tournament* (section 1.1.5) carries the same idea over into selection.

5.3 Tabu search

Tabu search (Glover, 1986)

Local search with a *short-term memory*: a *tabu list* of recently performed moves is kept (which is more efficient than storing whole solutions), and those moves are forbidden for the duration of the *tabu tenure*. In each step one moves to the **best non-forbidden neighbour**, even if it is worse than the current solution — this is how the algorithm escapes from a local optimum and avoids cycling.

- **Tabu tenure** T: static (a fixed number of iterations) or dynamic (drawn at random from a range).
- **Aspiration criterion**: the tabu is broken if the move leads to a solution better than the best one found so far (otherwise the tabu could block progress).
- **Medium-term memory**: rules steering the search into promising regions (intensification).
- **Long-term / frequency memory**: a counter of how often each element of a solution has been used; frequently visited configurations are penalised (diversification), or one restarts from a rarely visited region.

Applications: the TSP (edge exchanges, *ejection chains*), distribution problems, DNA sequencing, scheduling. Pros: escape from local optima, no cycling, it works in discrete as well as in continuous domains. Cons: many parameters, higher computational cost (traversing the whole neighbourhood).

5.4 Scatter search

A population metaheuristic focused on **diversity**. It maintains a small *reference set* R (on the order of tens of individuals) into which individuals are selected according to a combination of quality and mutual distance (the minimal distance of a newly added individual from the rest of R is maximised). The cycle: (1) *diversification* — a clever initialisation, (2) *subset generation* — typically all the pairs from R, (3) *combination* — arithmetic (or permutation) crossover, (4) *improvement* — local search, mutation, tabu search, (5) *updating* R. The reference set can be updated *incrementally* (one or a few individuals are added per generation), by a *complete renewal* every generation, or it can be built already in the inner loop — new individuals enter R immediately and are recombined with straight away. It is suitable for problems with a very expensive fitness (e.g. the black-box optimiser OptQuest).

5.5 Memetic algorithms

Meme and memetic algorithm

A *meme* (Dawkins, 1976) is a unit of cultural information spreading by imitation — a "gene of culture". A *memetic algorithm* (Moscato, 1989) is a hybrid method: **an evolutionary algorithm with a nested local search** that improves the individual solutions ("individual learning"). Synonyms: hybrid EA, genetic local search, Lamarckian or Baldwinian EA, cultural algorithms.

The essence: a local search nested inside the evolutionary cycle. The genetic operators carry information *between* the individual runs of the local searcher, so the overall search is far more efficient than a repeated restart of the local method (Krasnogor & Smith). As a meme one can use hill climbing, simulated annealing, tabu search, a combinatorial heuristic (2-opt) or a gradient method (backpropagation when training a neural network).

Lamarckism vs. Baldwinism

What is to be done with an improved individual $x' = \mathrm{LS}(x)$?

- **The Lamarckian way:** x' is inserted into the population together with its fitness — *the genotype is changed according to the phenotype.* From a Darwinian point of view this is "not

Algorithm 9 Memetic algorithm

```
initialise the population
while the termination condition is not met do
    evaluate the individuals in the population
    create the new population by the stochastic operators (selection, crossover, mutation)
    select a subset O of individuals that will undergo improvement
    for each x ∈ O do
        perform individual learning of x by means of a meme (a local searcher), with probability p for a time t
        process the result in the Lamarckian or the Baldwinian way
    end for
end while
```

kosher", but in practice it is faster.

- **The Baldwinian way:** the individual x is assigned the *fitness* of the improved x', but the genotype of x remains *unchanged*. This is Darwinistically correct; the fitness is interpreted as the "potential" of the individual (the *Baldwin effect*: the ability to learn favours an individual and evolution subsequently "catches up" by itself — genetic assimilation).

Lamarckism converges faster, but it reduces diversity more and may distort the landscape; Baldwinism preserves diversity and smooths the fitness landscape, but it is slower.

Design questions. Which individuals should be improved (all / the elite only / a random fraction p)? For how long (t steps)? How strongly (the depth of the local search)? Too much local search means unnecessary cost and a loss of diversity; too little brings no benefit.

Memetic computing. A generalisation: the memes are not known in advance but are themselves the object of the search.

- **Multi-meme MA:** the meme is encoded *as part of the genotype* of an individual; it is inherited, it is crossed (the offspring inherits the meme of the better parent, or at random) and it is used to improve that individual.
- **Meta-Lamarckian MA:** the memes have *their own population* and coevolve with the individuals; a meme is assigned to an individual for the improvement, the success of the individual is the reward for the meme, and a second evolutionary algorithm runs over the memes.
- The *memetic space* as a counterpart of the phenotype space: memes can be chosen by heuristics, made to compete according to their success, split into subpopulations, or controlled by a meta-memetic algorithm.

5.6 Parameter tuning and control

This is closely related: an EA has many parameters (the population size, p_C, p_M, the tournament size, the degree of elitism). They are not independent and an exhaustive search is not possible.

- **Tuning** — the parameters are determined *before* the run (trial and error, grid search, a meta-EA, iterated racing — irace).
- **Control** — the parameters change *during* the run:
  - A *fixed strategy* (deterministic): a dependence on time only (e.g. $\sigma(t) = 1 - 0.9\,t/T$, a decrease of p_M by 0.1 % per generation); the stochastic variant = simulated annealing.
  - *Adaptive*: according to feedback from the run (the quality of the solutions, the variance of fitness, the diversity) — e.g. the 1/5 rule, hypermutation upon a drop in fitness, adaptive refinement of the representation (increasing the number of bits per real number if the diversity, measured by the Hamming distance of the best and the worst individual, drops).
  - *Self-adaptive*: the parameters are part of the genome and evolve (ES, EP, multi-meme MA).

- The *population size* can also be controlled indirectly: every new individual is assigned a "lifetime" proportional to its fitness, and it dies once that expires. Better individuals live longer, and the population size is a derived dynamic quantity.

5.7 Exam summary

- Hill climbing: steepest ascent / first improvement / stochastic / with random restarts; the problems: local optima, plateaux, ridges.
- SA: the Metropolis criterion $P = \min(1, e^{-\Delta/T})$; the cooling schedule (geometric in practice, logarithmic for the convergence proof); for a fixed T it converges to the Boltzmann distribution $\propto e^{-f/T}$.
- Tabu search: a tabu list of moves, the tenure, the aspiration criterion, medium-term and long-term (frequency) memory; one always moves to the best non-forbidden neighbour, even if it is worse.
- Scatter search: a reference set combining quality and diversity, arithmetic combinations + improvement.
- Memetic algorithm = EA + local search over the individuals; **Lamarckism** (it changes the genotype, faster) vs. **Baldwinism** (it changes only the fitness, it preserves diversity). Memetic computing: memes in the genome or in a coevolving population.
- Parameter tuning vs. control; fixed / adaptive / self-adaptive strategies.

6 Applications of evolutionary algorithms

The last sentence of the topic enumerates four application areas; each forms one section of this chapter. What they have in common is that in each of them what decides is the **choice of representation and operators**, not the choice of a "better" evolutionary algorithm.

6.1 Evolution of rule-based and expert systems

6.1.1 Michigan vs. Pittsburgh

The goal is to learn a **set of rules** of the form `IF condition THEN action` from training data or from interaction with an environment (knowledge discovery, expert systems, robot control, reinforcement learning). There are two basic approaches:

	Michigan (Holland)	**Pittsburgh** (Smith)
individual	**a single rule**	**a whole set of rules**
solution	the whole population together	a single individual
fitness	the strength/accuracy of the rule (the reward has to be distributed)	the quality of the whole system (straightforward)
operators	standard, evolution runs only occasionally and on a part of the population	complex, dozens of operators over rule sets and over individual rules
advantages	on-line learning, a small space, natural for RL	an unambiguous fitness, no credit assignment
disadvantages	credit assignment; the rules have to cooperate, yet they compete at the same time	large individuals, expensive evaluation, a risk of bloat

6.1.2 Learning classifier systems (Michigan)

LCS (learning classifier system)

A system in which a population of simple rules (*classifiers*) forms an expert or control system as a whole. A classifier has a left-hand side (the condition) in the form of a string over $\{0, 1, *\}$ that is matched against the input, a right-hand side (an action or a classification class) and a **weight** / **strength** reflecting its success, which serves as its fitness. Evolution does not proceed generationally, but continuously and only on a part of the population.

The architecture of an LCS. The key point to realise is that *the expert system is the whole population*, not an individual. The system forms a loop:

1. The **detectors** convert the state of the environment into an input message.
2. The message is stored in the **message buffer** and matched against the left-hand sides of all the rules $\Rightarrow$ the *match set*.
3. Among the matching rules an **auction/competition** for the right to act takes place (according to the strength of the rule).
4. The winning rule writes its right-hand side either into the buffer (an internal message) or, through the **effectors**, into the environment as an action.
5. The environment returns a **reward**; this is distributed by the *bucket brigade* mechanism among the rules that led to it.
6. Occasionally, and only on a part of the population, a **GA** runs and renews the rules.

The problem of reactivity. A purely reactive system has no internal memory. Holland therefore introduced *messages*: besides an action, the right-hand side of a rule may write an internal message into the buffer, and the left-hand side has *receptors* for catching it. Chaining of rules arises in this way — the system acquires an internal state (and thereby computational power), but at the same time it brings the problem of how to distribute the reward over the whole chain.

Bucket brigade

The mechanism for distributing the reward in an LCS. A rule that wants to be applied must "pay" a part of its strength to the rule that activated it (as if it were buying the right to act). The reward from the environment goes to the last link of the chain and gradually "flows" back to the rules that led to it. In practice it is difficult to balance this economy of rules, which is why it is hardly ever used today.

ZCS (Wilson, 1994) — the "zero-level" LCS. A simplification: no internal messages, no complex redistribution.

- P — the population of rules; the input x from the detectors.
- M — the *match set*: the rules whose condition matches x.
- The action a is selected from M by the roulette wheel according to the strengths of the rules; $A \subseteq M$ — the *action set*: the rules advocating the chosen action.
- **Reinforcement:** a fraction β of the strength of the rules in A is taken away into a "bucket" B; after a reward r is received, every rule in A has $\beta r/|A|$ added to it and the rules of the previous action set A_{-1} have $\gamma B/|A_{-1}|$ added to them. The rules from $M \setminus A$ (they matched, but advocated a different action) are taxed.
- **The cover operator:** if no rule exists for the current input, one is generated ad hoc (stars are added at random into the condition and the action is chosen at random).

XCS (Wilson, 1995) — accuracy-based fitness. The weaknesses of ZCS: it does not evolve a *complete* system of rules covering all situations; rules at the beginning of chains are rewarded

rarely and die out; rules leading to small but correctly predicted rewards die out as well. The cause is that the strength served both as the fitness for the GA and as the criterion for choosing the action. Wilson's idea: the *prediction* should estimate the reward, but *evolution* should prefer **reliable (accurate)** classifiers.

XCS — a classifier as a quintuple

(c, a, p, ϵ, F): the condition, the action, the reward prediction p, the prediction error ϵ, the fitness F. The accuracy is computed as a decreasing function of the error, $\kappa = \alpha\,(\epsilon/\epsilon_0)^{-\nu}$ (for $\epsilon > \epsilon_0$, otherwise $\kappa = 1$; α, ϵ_0, ν are parameters); the fitness is the *relative* accuracy within the action set. The prediction and the error are updated by the rules of Q-learning.

A run of XCS: according to the input the match set M is assembled and split into action sets A_i according to the actions; for each action the payoff is predicted as the fitness-weighted average of the predictions; the action is chosen either by the maximum (exploitation) or by the roulette wheel (exploration); it is performed; the reward is distributed into the action set of the previous step (an update of p using $r + \gamma \max_a p_a$, then ϵ, then F); the GA runs as a **niche** GA — only inside the action set, not over the whole population.

The result: XCS links LCS with reinforcement learning (Q-learning as the credit assignment mechanism), evolves a **complete and minimal** input–output map (not only an optimal policy, but also a compact, comprehensible description) and has a number of practical applications (knowledge discovery, control, bioinformatics).

Today's family of LCS. Depending on the task one distinguishes: *XCS* (reinforcement learning, reward prediction), **UCS** (*sUpervised Classifier System*) — the variant for supervised learning, where the correct class is known, so that classification accuracy is measured directly instead of a reward prediction, and *XCSF* for the approximation of continuous functions (the right-hand side is a linear model, not a constant). Nowadays the conditions are commonly written not only over $\{0, 1, *\}$ but also with intervals for real-valued attributes, which makes LCS a fully-fledged knowledge discovery tool — with the advantage that the result is a **readable set of rules**.

6.1.3 The Pittsburgh approach and the GIL system

An individual = a whole set of rules, i.e. a complete classification system. The evaluation is more complicated (rule priorities, conflicts, false positive and false negative answers) and there is a whole range of operators, at several levels. The emphasis is on a rich domain representation (sets, enumerations, intervals).

GIL is the classical representative: it is meant for binary classification, and an individual classifies implicitly into one class (the rules have no right-hand side). An individual is a *disjunction of complexes*, a complex is a *conjunction of selectors* (one for each variable) and a selector is a *disjunction of values* from the domain of the given variable, encoded by a bit mask. For example

$$\big((X{=}A_1) \wedge (Z{=}C_3)\big) \vee \big((X{=}A_2) \wedge (Y{=}B_2)\big) \;\equiv\; [\,001|11|0011 \;\vee\; 010|10|1111\,].$$

The operators work at three levels:

- the *individual*: exchange of rules, copying a rule, generalising a rule, deleting a rule, specialising a rule, including a positive example;
- the *complex*: splitting a complex on one selector, generalising a selector (replacing it by $11\ldots1$), specialising it, including a negative example;
- the *selector*: mutation $0 \leftrightarrow 1$, widening $0 \to 1$, narrowing $1 \to 0$.

These operators correspond directly to the operations of inductive learning (generalisation and specialisation), so GIL is in fact an evolutionarily driven inductive algorithm.

6.1.4 A reminder: classification metrics

From the confusion matrix (TP, FP, FN, TN) one computes accuracy $= (TP + TN)/(TP + TN + FP + FN)$, sensitivity $= TP/(TP + FN)$ and specificity $= TN/(TN + FP)$. The fitness of a rule-based system is usually a combination of accuracy, coverage and simplicity (the number of rules) — so it is naturally a multi-objective problem.

6.2 Neuroevolution

Neuroevolution

"Neuroevolution is a method for modifying neural network weights, topologies, or ensembles in order to learn a specific task. Evolutionary computation is used to search for network parameters that maximize a fitness function that measures performance in the task. Compared to other neural network learning methods, neuroevolution is highly general — it allows learning without explicit targets, with non-differentiable activation functions, and with recurrent networks." (Miikkulainen, 2017)

What can be evolved: the weights, further parameters (activation functions, learning constants), the architecture (the topology, the connections), the architecture and the weights at the same time, or the parameters of the learning algorithm itself. The main domain is **reinforcement learning** and robotics, where target outputs for supervised learning are not available.

6.2.1 Evolution of the weights

Straightforward: the weights are encoded into a vector of fixed length and a real-valued GA, an ES or DE with the standard operators is used. Properties:

- Slower than specialised gradient methods (backpropagation) for supervised learning.
- + Easy parallelisation, no gradient needed (it works for non-differentiable and recurrent networks), usable for RL.
- **The problem of competing conventions** (the permutation problem): the same network function corresponds to many different genotypes (permutations of the hidden neurons), so crossing two functionally identical parents often produces a non-functional offspring.
- A recent revival of interest: the evolution of weights is competitive even for large networks if evaluation on minibatches is used.

6.2.2 Evolution of the architecture

The fitness of an architecture = an estimate of the performance of the network: the network has to be built, initialised and trained (by backpropagation or by another EA), preferably several times — which is why the fitness evaluation is expensive and noisy.

- **Direct encoding**: the connection structure as a binary matrix; an individual is either the linearised matrix (standard operators) or the matrix itself (special 2D operators). Simple, but it does not scale (the size of the genome grows quadratically).
- **Grammar encoding** (Kitano, 1990): the connection matrix is generated by a simple 2D formal grammar and the *rules of the grammar* are evolved. Compact (logarithmic), it allows repeated motifs, but it is indirect and harder to tune.
- **Cellular encoding** (Gruau): an individual is a **GP program** describing how to *grow* the network from a single cell by means of the operations: add a neuron, split a neuron serially, split it in parallel, reconnect a synapse, change a weight. A developmental approach.
- **Simulated growth** of connections in a 2D plane — early attempts in evolutionary robotics; it did not scale.

GNARL (Angeline et al., 1993). Evolutionary programming over graphs: the input and output neurons are fixed, the hidden neurons and the connections are evolved; the architecture and the weights *at the same time*. Two kinds of mutation: *structural* (add a neuron without connections, add a connection with zero weight, delete a neuron, delete a connection) and *parametric* (biased mutation of a weight). The strength of the mutation is controlled by a *temperature* (a simulated annealing approach); structural changes should be small and not too destructive. Selection: the better half of the population becomes the parents. Crossover is not used (EP).

6.2.3 NEAT

NEAT (NeuroEvolution of Augmenting Topologies, K. Stanley, 2002)

The network is represented as a **list of edges**; every edge gene contains: the input and the output neuron, the weight, an *enabled/disabled* flag and a **global innovation number**, which is assigned when the given structural novelty first arises.

Three key ideas:

1. **Innovation numbers solve the problem of competing conventions.** Only genes with *matching innovation numbers* are crossed (so they have a common evolutionary origin); genes that are surplus in one of the parents (*disjoint* and *excess*) are taken over from the fitter parent. Crossover thus does not disrupt the structure — no "headless chicken".
2. **Speciation (niching) protects innovations.** On the vectors of innovation numbers a similarity measure is defined:
$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{\Delta W},$$
where E (*excess*) is the number of genes whose innovation number lies *beyond* the highest innovation number of the other genome, D (*disjoint*) is the number of genes missing in the other parent *inside* the common range of innovation numbers, $\overline{\Delta W}$ is the average difference of the weights of genes with *matching* innovation numbers and N is the size of the larger genome (for normalisation). Networks with δ below a threshold form one *species*; fitness is computed as shared (relative within the species). A new topology thus has time to "fine-tune its weights" before established solutions crowd it out — otherwise a structural novelty that initially worsens the fitness would die out immediately.
3. **Gradual growth of complexity** (*complexification*): the population starts from minimal networks (inputs and outputs only) and grows only through the mutations *add a connection* and *add a neuron* (which splits an existing connection: the original one is disabled and two new ones arise). The search therefore starts in the space of the lowest dimension and increases it only when it pays off.

6.2.4 HyperNEAT and CPPNs

[♦ beyond the slides: the slides give it one line, CPPNs not at all]

HyperNEAT extends NEAT by two ideas. *The substrate*: the weights of the network are not encoded as a vector of numbers, but *are generated by a function* $S : \mathbb{R}^4 \to \mathbb{R}$ which receives the coordinates of two neurons in the plane and returns the weight of their connection (hence "hyper"). *A CPPN* (compositional pattern-producing network): the function S is represented by a network in the form of a DAG whose nodes compute various functions from a dictionary (sigmoid, Gaussian, sine, absolute value); this CPPN is evolved by NEAT itself. The benefit: representing the weights by a function makes it possible to express **symmetries and repeated motifs** and to generate networks with millions of connections from a small genome.

6.2.5 Neuroevolution in the era of deep networks

[♦ beyond the slides]

NAS (neural architecture search) looks for the architecture of deep networks and is described along three axes: the *search space* (a sequence of layers, a DAG of repeated cells, or a subarchitecture of one large "supernetwork"), the *search strategy* (evolutionary algorithms, random search as a surprisingly strong baseline, Bayesian optimisation, reinforcement learning, gradient-based DARTS) and *performance estimation* (shortened training, weight sharing). Concrete systems: **ENAS** (weight sharing among the candidates), **DeepNEAT** (a node of the chromosome is a whole layer) and **CoDeepNEAT** (coevolution of a population of modules and a population of "blueprints"). A separate line is **ES for RL** (OpenAI 2017): a simple evolution strategy *with mutations only* over millions of weights, trivially parallelisable — the nodes exchange only the seeds of the random number generator.

6.3 Combinatorial problems and constraint handling

Evolutionary algorithms are a popular tool for NP-hard combinatorial problems: they do not guarantee the optimum, but they provide a good solution in a reasonable time and adapt easily to the additional (possibly quite ugly) constraints of a real problem.

6.3.1 The knapsack problem

The task. A knapsack of capacity $C_{\max}$, N items with values v_i and volumes c_i. Maximise $\sum_i x_i v_i$ subject to $\sum_i x_i c_i \leq C_{\max}$, $x_i \in \{0, 1\}$.

The encoding is trivial — a bit map, e.g. 0110010 means "take items 2, 3 and 6". The standard operators (crossover, bit-flip mutation) work. **The problem is the fitness**: individuals may violate the capacity. We have two conflicting criteria: to maximise $\sum v_i$ and at the same time to satisfy $\sum c_i \leq C_{\max}$.

6.3.2 Three general strategies for constraints

Constraint handling in EAs

1. Penalty: infeasible solutions stay in the population, but their fitness is reduced, e.g. $f'(x) = \sum_i x_i v_i - \alpha \max\left(0, \sum_i x_i c_i - C_{\max}\right)$. The choice of α is crucial: too small ⇒ the population fills up with infeasible solutions, too large ⇒ the possibility of passing through the infeasible region towards a promising solution is lost. Dynamic (α growing in time) or adaptive penalties are also used.
2. Repair: an infeasible solution is repaired (for the knapsack: throw out the items with the worst ratio v_i/c_i until it fits). The repair can be performed only for the computation of fitness (the Baldwinian way) or written permanently into the genotype (the Lamarckian way).
3. Decoder / closed operators: the encoding is designed so that *every* genotype represents a feasible solution. For the knapsack: a bit 1 means "add the item if it still fits", otherwise skip it. Elegant and effective; the drawback is the ambiguity of the mapping (several genotypes give the same phenotype) and its poorer locality.

A fourth possibility: to regard the violation of the constraint as a **second criterion** and to use a multi-objective EA (section 6.4) — the result is then a whole Pareto front of trade-offs between the quality of the solution and the degree of constraint violation.

6.3.3 The travelling salesman problem

The TSP: N cities, find the shortest round tour. The fitness is trivial (the length of the tour); **the problem is the encoding and above all the crossover**.

Representations.

- **Adjacency**: at position i there is city j precisely if there is an edge $i \to j$. For instance, (248397156) corresponds to the tour 1−2−4−3−8−5−9−6−7. Every tour has a unique representation, but some lists are not valid tours. Classical crossover does not work; what is interesting is that schemata make sense here (e.g. $(*3*\ldots)$ = all the tours with the edge 2–3). *Do not use.*
- **Ordinal** (*buffer*): a list of cities is maintained, gene i is the *position* of the city in the current list, and after being used the city is removed from the list. For the list (123456789) the tour 1−2−4−3−8−5−9−6−7 is encoded as (112141311). The motivation was to make standard one-point crossover possible — and it does produce valid tours, but the result has nothing in common with the parents. *Also not to be used.*
- **Path (permutation)**: the tour 5−1−7−8−9−4−6−2−3 is (517894623). The most natural one; it requires special operators — PMX, OX, CX, ER (section 1.3).
- **Matrix**: a binary matrix in which a 1 at position (i, j) means either the edge $i \to j$ or (more often) that i precedes j. Special operators: *conjunction* (a bitwise AND of the two parents and a random completion of the missing edges), *disjunction* (a split into quadrants, two of which are discarded, the contradictions are removed and the rest is filled in at random).

Initialisation. Random permutations, or constructive heuristics: *nearest neighbour* (start in a random city and always go to the nearest unvisited one) and *edge insertion* (for a partial tour T choose the nearest city c outside T, find the edge k–j in T minimising $d(k, c) + d(c, j) - d(k, j)$, and replace that edge by the pair k–c, c–j).

Mutation. Inversion of a segment (the most important one — it corresponds to a 2-opt step), inserting a city elsewhere, shifting a subtour, swapping two cities, swapping subtours, the heuristic mutations 2-opt and 3-opt (take two edges and reconnect four cities differently). A combination of an ES or a GA with "clever mutations" of the 2-opt kind is very effective in practice — de facto a memetic algorithm.

6.3.4 SAT

[♦ beyond the slides: the slides do the knapsack, the TSP and scheduling; SAT comes from Michalewicz]

The task. A formula $f : \mathbb{B}^n \to \mathbb{B}$ in CNF; find an assignment x with $f(x) = 1$. 2-SAT is in P, 3-SAT and above is NP-complete.

Fitness. f itself is unusable (it takes only the values 0/1, so the landscape is a "needle in a haystack"). The standard is the **number of satisfied clauses**; the improvements are weighting the problematic clauses and a *refining function* — a regularisation term that distinguishes individuals with the same number of satisfied clauses and thus helps off the plateaux.

The representation is usually a plain bit string. The *real-valued* variant is interesting: the variable x_j is replaced by a real y_j (true = 1, false = −1), the literals are rewritten as $x \mapsto (1 - y)^2$ and $\neg x \mapsto (1 + y)^2$, and the expression is *minimised*. The value of a literal is therefore a *penalty*, and that is why a **disjunction** (a clause) is rewritten as a **product** and a **conjunction** (the formula) as a **sum**. The classical local heuristic is **WalkSAT**; the combination EA + WSAT is a typical memetic solution.

6.3.5 Scheduling and production planning

A school timetable. An individual is a timetable in the form of a matrix (rows = teachers, columns = classes, values = subject codes). Mutation shuffles the subjects, crossover exchanges the better rows between individuals. The fitness combines the quality of the individual rows and soft criteria (gaps, uniformity). **Hard constraints** (a teacher cannot teach two classes at once) have to be respected *directly in the operators* — otherwise far too many infeasible individuals arise.

Job shop scheduling. Products consist of parts, for each part there are several production plans on the machines, and resetting a machine costs time; the fitness is the total production time (the *makespan*). The encoding is critical: a *permutation-based* individual (only the order of the products, the rest determined by a decoder) allows the TSP crossovers to be used, but turns out to be not very effective — the difficult part is solved by the decoder. A *direct* encoding of the whole plan is more powerful but requires specialised operators.

6.4 Multi-objective optimisation

6.4.1 Dominance and the Pareto front

Instead of a single fitness we have a vector of criteria $f_1, \ldots, f_k$ (for simplicity we minimise everything). The criteria are usually in conflict (cost vs. quality, accuracy vs. the size of the model), so there is no single best solution.

Dominance

For individuals x, y we define:

- x **weakly dominates** y ($x \preceq y$) if and only if $f_i(x) \leq f_i(y)$ for all $i = 1, \ldots, k$.
- x **dominates** y ($x \prec y$) if and only if $x \preceq y$ and there exists a j with $f_j(x) < f_j(y)$.
- x and y are **incomparable** if neither $x \prec y$ nor $y \prec x$.

The relation $\prec$ is a *strict* partial order — it is irreflexive and transitive, but not total: that is why in general there is a whole set of mutually incomparable optima.

Pareto optimality and the Pareto front

A solution x^* is *Pareto-optimal* if it is not dominated by any feasible solution. The set of all Pareto-optimal solutions in the space of the variables is the *Pareto set*, and its image in the space of the criteria is the **Pareto front**. An *approximation* of the Pareto front by a population is the goal of multi-objective EAs.

The algorithm has a twofold goal: (a) **convergence** — to get as close as possible to the true front; (b) **diversity** — to cover the front evenly over its whole extent. This is exactly why EAs are ideal for this task: they work with a population and can therefore return a whole approximation of the front in a single run.

Example. The points $A(1, 10)$, $B(2, 7)$, $C(4, 5)$, $D(8, 2)$, $E(5, 8)$ (both criteria minimised). E is dominated by C ($4 \leq 5$, $5 \leq 8$, at least one of them strict) — so E does not belong to the front. The points A, B, C, D are mutually incomparable and form the Pareto front.

6.4.2 Scalarisation — the simple solutions

- **Linear scalarisation**: $f = \sum_i w_i f_i$, followed by a standard single-objective EA (referred to as an SOEA in the MOEA context). The drawbacks: we do not know how to choose the weights; one run gives one point of the front; and, crucially, **non-convex parts of the front can never be found by linear scalarisation**.
- **ε-constraint**: minimise f_1 subject to the constraints $f_i \leq \varepsilon_i$ for $i \geq 2$. It can find non-convex parts as well, but the constants ε_i have to be chosen and a constrained problem has to be solved.
- **Chebyshev scalarisation**: minimise $\max_i \lambda_i |f_i(x) - z_i^*|$, where z^* is a reference (ideal) point. Unlike the linear one, it can reach any point of the front, including the non-convex parts.

6.4.3 The historical algorithms

VEGA (Schaffer, 1985) — one of the first MOEAs. The population of N individuals is sorted for each criterion f_i and the N/k best according to f_i are selected; these subsets are merged, crossed and mutated. The drawbacks: it is difficult to maintain diversity and it converges to the extreme points optimal for the individual criteria (the "middles" of the front are lost).

NSGA (1994) — the first use of dominance directly as the fitness. The population is split into fronts: F_1 = the non-dominated individuals of P, F_2 = the non-dominated individuals of $P \setminus F_1$, F_3 = the non-dominated ones of $P \setminus (F_1 \cup F_2)$, and so on. The individuals of the first front receive a "dummy" fitness, which is divided by a *niche factor* $\sum_j sh(i,j)$, where $sh(i,j) = 1 - (d(i,j)/d_{\text{share}})^2$ for $d(i,j) < d_{\text{share}}$ and 0 otherwise. The second front receives a fitness lower than the worst individual of the first front, again divided by the niche factor, and so on. The drawbacks: the need to set d_{share}, the absence of elitism, and the computational complexity $O(kN^3)$.

6.4.4 NSGA-II

NSGA-II (Deb et al., 2000)

It repairs the shortcomings of NSGA. Three pillars: **(1) fast non-dominated sorting** into fronts (complexity $O(kN^2)$), **(2) the crowding distance** instead of the parameter d_{share}, **(3) elitism** — the parents and the offspring are merged and sorted, and the better half forms the new population.

Crowding distance

For each front separately: for each criterion m sort the individuals according to f_m, assign ∞ to the extreme individuals and add to the others the normalised distance of their neighbours:

$$d_i = \sum_{m=1}^{k} \frac{f_m(i+1) - f_m(i-1)}{f_m^{\max} - f_m^{\min}}.$$

It expresses the "perimeter of the box" formed by the nearest neighbours — a large value means a sparsely occupied region, which we want to preserve.

Crowded comparison operator

An individual i is better than j if and only if (a) i lies in a *lower* front ($rank_i < rank_j$), or (b) the fronts are the same and i has a *larger* crowding distance. No scalar fitness is therefore computed at all — only these two numbers are compared, typically in a binary tournament.

Algorithm 10 NSGA-II (one generation)

1: $R_t \leftarrow P_t \cup Q_t$ ▷ parents and offspring, $2N$ individuals in total
2: $(F_1, F_2, \dots) \leftarrow$ non-dominated sorting of R_t
3: $P_{t+1} \leftarrow \emptyset$; $i \leftarrow 1$
4: **while** $|P_{t+1}| + |F_i| \le N$ **do**
5: compute the crowding distance in F_i; $P_{t+1} \leftarrow P_{t+1} \cup F_i$; $i \leftarrow i + 1$
6: **end while**
7: sort F_i in decreasing order of the crowding distance and fill P_{t+1} up to N individuals
8: $Q_{t+1} \leftarrow$ tournament selection (crowded comparison) + crossover (SBX) + mutation

A numerical example of the crowding distance. The front $A(1,10)$, $B(2,7)$, $C(4,5)$, $D(8,2)$; the ranges: $f_1 \in [1,8]$ (a span of 7), $f_2 \in [2,10]$ (a span of 8). The extreme points A and D receive ∞ (they are extreme in both criteria).

$$d_B = \frac{f_1(C) - f_1(A)}{7} + \frac{f_2(A) - f_2(C)}{8} = \frac{4-1}{7} + \frac{10-5}{8} = 0.43 + 0.63 = 1.05,$$

$$d_C = \frac{f_1(D) - f_1(B)}{7} + \frac{f_2(B) - f_2(D)}{8} = \frac{8-2}{7} + \frac{7-2}{8} = 0.86 + 0.63 = 1.48.$$

If one of B, C has to be chosen, C wins — it lies in a more sparsely occupied part of the front.

NSGA-III. For problems with many criteria ($k \geq 4$, *many-objective*), where the crowding distance fails (almost everything is non-dominated). The crowding distance is replaced by a set of evenly distributed **reference points** (their number roughly corresponds to the population size, $N = \binom{p+k-1}{p}$ for p divisions). After the non-dominated sorting those reference directions that have the fewest solutions assigned to them are filled preferentially; if a direction has no solution, the individual with the smallest perpendicular distance from the corresponding reference vector survives.

6.4.5 Other important algorithms

[♦ beyond the slides: the slides stop at NSGA-II]

SPEA2 uses an external **archive** of non-dominated solutions; the fitness of an individual is the sum of the *strengths* of all the individuals that dominate it (strength = the number of individuals the given individual dominates), plus a density derived from the distance to the σ-th nearest neighbour. **MOEA/D** decomposes the problem into μ *scalar* subproblems with evenly distributed weight vectors (Chebyshev scalarisation); every individual corresponds to one subproblem and cooperates only with its neighbourhood — hence the low complexity and good scalability. **SMS-EMOA** selects directly by the contribution to the *hypervolume* (the best-founded theoretically, but exponentially expensive in the number of criteria). **NSGA-III** replaces the crowding distance by evenly distributed **reference points** and targets many-objective problems ($k \geq 4$), where the crowding distance fails.

6.4.6 Metrics of approximation quality

[♦ beyond the slides]

Hypervolume (the S-metric)

The volume of the region dominated by the front found and bounded by a *reference point* r (chosen so that it is dominated by all the solutions). The only common metric that is *strictly monotone with respect to dominance* and that does not require knowledge of the true Pareto front.
Example: the front $\{(2,7),(4,5),(8,2)\}$, $r = (10,10)$. By vertical strips: $(8,2)$ gives $2 \cdot 8 = 16$, $(4,5)$ gives $4 \cdot 5 = 20$, $(2,7)$ gives $2 \cdot 3 = 6$; in total $HV = 42$. Adding a dominated point does not change the value.

Furthermore *GD* (the average distance of the points found from the true front — it measures convergence), *IGD* (the other way round; it measures both convergence and coverage) and *spread* (the evenness of the distribution); both GD and IGD require knowledge of the true front.

6.4.7 An overall comparison of MOEAs

algorithm	selection principle	diversity	note
VEGA	subpopulations by f_i	none	converges to the extremes
NSGA	fronts + dummy fitness	fitness sharing (d_{share})	no elitism, $O(kN^3)$
NSGA-II	fronts + crowding	crowding distance	elitism, $O(kN^2)$, standard
NSGA-III	fronts + reference points	reference directions	many-objective
SPEA2	strength of dominators	σ-th neighbour, truncation	external archive
MOEA/D	scalarised subproblems	even weight vectors	neighbourhoods, scales
SMS-EMOA	hypervolume contribution	implicit	best-founded, expensive

6.5 Exam summary

- **Michigan**: an individual = a single rule, the solution is the whole population, the reward has to be distributed (credit assignment). **Pittsburgh**: an individual = a whole set of rules, the fitness is straightforward, the operators are complex (GIL).

- Holland's LCS: conditions over $\{0, 1, *\}$, internal messages and receptors, the bucket brigade (a rule pays its predecessor for the right to act).
- ZCS: no messages; the match set $M \to$ roulette wheel selection of the action $\to$ the action set A; reinforcement of A and A_{-1}, a tax for $M \setminus A$; the cover operator.
- XCS: a classifier (c, a, p, ϵ, F), **fitness based on the accuracy of the prediction** $\kappa = \alpha(\epsilon/\epsilon_0)^{-\nu}$, updates by Q-learning, a niche GA in the action set; it evolves a complete and minimal input–output map. Variants: UCS (supervised learning), XCSF (function approximation).
- Be able to explain why accuracy as the fitness solves the shortcomings of "strength-based" systems.
- What is evolved: the weights / the topology / both / the hyperparameters of the learning. The advantages of EAs: no gradient, recurrent networks, RL, parallelisation.
- The problem of competing conventions (the permutation problem) — without precautions, crossing networks is destructive.
- Encodings of the architecture: direct (a matrix), grammar-based (Kitano), cellular (Gruau, a GP program for growing the network); GNARL = EP over graphs, structural as well as parametric mutations controlled by a temperature.
- NEAT: a list of edges with **innovation numbers** (only matching genes are crossed), **speciation** according to the similarity of the genomes with shared fitness (protection of new topologies), **complexification** from a minimal network.
- HyperNEAT: a substrate $S : \mathbb{R}^4 \to \mathbb{R}$ generating the weights from the coordinates of the neurons, represented by a CPPN (a DAG with various activation functions) evolved by NEAT; it captures symmetries and regularities.
- NAS: the space (linear / a DAG of cells / a supernetwork), the strategy (EA, RL, Bayesian, gradient-based), the performance estimate (weight sharing — ENAS); DeepNEAT and CoDeepNEAT (modules + blueprints); ES for deep RL.
- The knapsack: a bit map, trivial operators, a problem with the capacity constraint. Three strategies for constraints: **a penalty** (the choice of the weight α), **a repair** (Lamarckian/Baldwinian), **a decoder** ("add it if it fits" — every genotype is feasible); a fourth possibility = a multi-objective formulation.
- The TSP: the fitness is trivial, the problem is the encoding. Do not use the adjacency and the ordinal representation; use the permutation one + PMX/OX/CX/ER, or possibly the matrix one. Initialisation: nearest neighbour, edge insertion. Mutation: inversion (2-opt).
- SAT: the fitness = the number of satisfied clauses (f alone is not enough), weighting of the clauses, a refining function; the representations are bit-based, real-valued, clause-based, path-based; WalkSAT as a local heuristic. With the real-valued representation ($x \mapsto (1-y)^2$, $\neg x \mapsto (1+y)^2$, minimisation) beware: the values are *penalties*, so a disjunction (a clause) is a **product** and a conjunction (the formula) a **sum**.
- Scheduling: a direct matrix encoding, hard constraints to be handled in the operators, soft ones in the fitness; job shop — a permutation with a decoder vs. a direct encoding.
- The general rule: in combinatorial problems what matters is the encoding and operators that respect the structure of the problem; hybridisation with a local heuristic (2-opt, WSAT) is almost always advantageous.
- Dominance ($x \prec y$: not worse in all the criteria and better in at least one), the Pareto set and the Pareto front; the twofold goal = convergence + diversity.
- Scalarisation: linear (simple, but it will not find the non-convex parts of the front), ε-constraint, Chebyshev.
- NSGA-II: non-dominated sorting into fronts, the crowding distance $d_i = \sum_m (f_m(i+1) - f_m(i-1))/(f_m^{\max} - f_m^{\min})$, the extreme points ∞; the crowded comparison (a lower front, and in case of a tie a larger crowding distance); elitism through the merge $P_t \cup Q_t$. Be able to compute the crowding distance on an example.
- SPEA2 (an archive, the strength of the dominators, the density via the σ-th neighbour), MOEA/D (Chebyshev scalarisation, weight vectors, neighbourhoods), SMS-EMOA (the hypervolume contribution), NSGA-III (reference points for many-objective problems).
- Hypervolume: the volume dominated by the front with respect to a reference point; the only

common metric that is monotone with respect to dominance and that does not need the true front. Furthermore GD, IGD, spread, the ε-indicator.

Overall comparison and typical examination questions

A map of the field

- **Evolutionary algorithms** (a population, fitness, crossover, mutation, selection): genetic algorithms (binary, integer, permutation, real-valued), evolution strategies, differential evolution, evolutionary programming, genetic programming (tree-based, linear, Cartesian, grammatical evolution), learning classifier systems, estimation of distribution algorithms (EDA).
- **Nature-inspired methods without evolution**: swarm algorithms (PSO, ACO, ABC, firefly).
- **Metaheuristics not inspired by biology**: tabu search, scatter search, novelty search, simulated annealing, memetic algorithms.
- **Application areas** with their own representations and operators: combinatorial optimisation, neuroevolution, reinforcement learning, multi-objective optimisation, AutoML/NAS.

An overall comparison table of the algorithms

algorithm	representation	variation	selection	what to remember
SGA	binary string	1-point crossover, bit-flip	roulette wheel, generational	schema theory, BBH
ES	$\mathbb{R}^n + \sigma$	Gaussian mutation, recombination	random parents, $(\mu, \lambda)/(\mu+\lambda)$	the 1/5 rule, self-adaptation
CMA-ES	$\mathbb{R}^n$, the model $\mathcal{N}(m, \sigma^2 C)$	sampling from the distribution	rank-based, the μ best	evolution paths, rank-1/rank-μ
DE	$\mathbb{R}^n$	$\vec{x}_a + F(\vec{x}_b - \vec{x}_c)$, binomial crossover	parent–offspring duel	the scale from the population
EP	domain-specific (an automaton)	mutation only	tournament of parents and offspring	genotype = phenotype
GP	syntax tree	subtree exchange, mutation	tournament	bloat, ADFs
PSO	$\mathbb{R}^n$ + velocity	the equations of motion	none (memory only)	w, c_1, c_2, gbest/lbest
ACO	construction of a path	pheromone + heuristic	implicit	$\tau^\alpha \eta^\beta$, evaporation
SA	a single solution	a neighbour	Metropolis	the cooling schedule
MA	arbitrary	EA operators + local search	arbitrary	Lamarck vs. Baldwin
NSGA-II	arbitrary	SBX + polynomial mutation	fronts + crowding	elitism, the crowding distance
NEAT	list of edges	structural mutations, crossover by ID	shared fitness within a species	innovation numbers, speciation

Cross-cutting topics — what is asked across the questions

- **Exploration vs. exploitation** — how each algorithm handles it: a GA (crossover vs. the selection pressure), an ES (σ and the 1/5 rule), SA (the temperature), PSO (w and the topology), ACO (q_0, evaporation), tabu search (the tabu list), novelty search (exploration only).
- **Maintaining diversity** — fitness sharing, crowding, speciation, the island model, non-random mating, restarts, hypermutation.
- **Parameter adaptation** — tuning vs. control; fixed, adaptive and self-adaptive strategies; where each of them occurs in which algorithm.

- **Constraint handling** — penalties, repair, decoders, closed operators, a multi-objective formulation.
- **Lamarckism vs. Baldwinism** — memetic algorithms, the repair of individuals, neuroevolution with the retraining of the weights.
- **The theoretical background** — the schema theorem and its criticism, Vose's model, Markov chains and the convergence of an elitist GA, No Free Lunch, the convergence of SA.

Typical examination questions

1. Describe the general scheme of an evolutionary algorithm and list the types of selection together with their properties. Compute roulette wheel selection for a given population.
2. State and prove the schema theorem. What are the order and the defining length? What are the weaknesses of the theorem?
3. Explain the building blocks hypothesis and implicit parallelism. What is a deceptive problem?
4. Describe Vose's model of the SGA: what are the vectors p and s, what does the operator $\mathcal{F}$ do, what does $\mathcal{M}$ do, and what fixed points do they have? What does the finite-population model as a Markov chain look like?
5. What are estimation of distribution algorithms (EDA)? What is their relation to Vose's view of a GA and how do UMDA/PBIL differ from BOA?
6. What is the difference between a (μ, λ)- and a $(\mu + \lambda)$-ES? Explain the 1/5 rule and the self-adaptation of σ. How does CMA-ES work?
7. Describe one step of differential evolution including the formulas and a numerical example. What are the mutation variants?
8. How does tree-based genetic programming work? What is bloat and how is it fought? What are ADFs for?
9. Explain grammatical evolution and Cartesian GP — what is the genotype and what is the phenotype?
10. What is coevolution, what types does it have and what pathologies? How is fitness evaluated?
11. What is open-ended evolution, what are its hallmarks and why do artificial systems "get stuck"? How does novelty search work and why does it pay off to throw the goal away in deceptive problems?
12. Explain the iterated prisoner's dilemma, the TFT strategy and the properties of successful strategies.
13. Write down the PSO equations and explain the meaning of the individual terms. What is the difference between the gbest and the lbest topology?
14. Describe ACO: the transition rule, the pheromone update, the difference between AS and ACS.
15. Explain simulated annealing, the Metropolis criterion and the cooling schedule. When does it converge?
16. What is a memetic algorithm? Explain the difference between Lamarckian and Baldwinian learning.
17. Compare the Michigan and the Pittsburgh approach. How does XCS work and why is its fitness based on accuracy?
18. Describe NEAT: what are the innovation numbers and the speciation for? What does HyperNEAT add?
19. How is the TSP solved by an evolutionary algorithm? Describe PMX, OX, CX and ER.
20. How are constraints handled in an EA (using the knapsack problem as an example)?
21. Define dominance and the Pareto front. Describe NSGA-II and compute the crowding distance on an example. What is the hypervolume?

Concluding summary

- Evolutionary and swarm algorithms are **robust metaheuristics** for problems where exact and gradient methods fail: black-box, non-differentiable, discrete, structured, multi-objective and

dynamic problems.

- Their strength does not lie in one "best" algorithm (No Free Lunch), but in **the ability to adapt the representation and the operators to the given domain** and in the possibility of **hybridisation** with local heuristics and domain knowledge.
- The theory (schemata, Vose's model, convergence theorems, runtime analysis) provides intuition and asymptotic guarantees, but the practical design remains to a large extent an experimental discipline.